EE627 Final Project
Stacked Ensemble Music Recommender

Jude Eschete

Source code, all submission CSVs, and run logs:

`https://github.com/JEschete/EE627`

May 4, 2026

Executive Summary

The final project's stacked ensemble [12] reached a Kaggle public score of **0.922**, an improvement of +0.011 over the midterm production submission (0.911) and +0.008 over the strongest individual base model from Homework 9 (0.914). The final submission is a logistic-regression meta-learner trained on the 6,000 `test2_new.txt` labels, fed by twenty-three feature columns drawn from every prior assignment in the semester: Project 1's evidence/genre heuristics, Homeworks 3–7's tuned linear baselines, Homework 8's PySpark MF, Homework 9's seven PySpark classifiers, the midterm's BPR neural ranker, and one new orthogonal base model (item-item collaborative filtering) added in this final.

Five iterations of the meta-learner were built and submitted. The most important finding is empirical: between v1 and v3 the Kaggle score moved 0.918 $\rightarrow$ 0.919 $\rightarrow$ 0.919 despite three rounds of meta-learner refinements (regularization tuning, calibration, NDCG@3 cross-validation). The plateau was broken in v4 by adding an item-item collaborative filtering signal that operates on the raw rating-co-occurrence matrix instead of on the same 32-feature representation every other model used. v4 jumped to 0.922. v5 confirmed v4 by tying it with a tuned multi-variant iicf stack, which validates that the item-item CF signal was the binding constraint, not meta-learner sophistication.

1 Process and Techniques

Hardware note. The pipeline runs on a single workstation; CUDA is used opportunistically. Production runs were timed on an NVIDIA RTX 5080 (16 GB, CUDA 12.x), with the GPU accelerating only two stages: the rank-64 matrix-factorization training (HW7-style SGD, $\sim$20 s on the 5080) and the BPR neural ranker training ($\sim$90 s). Both modules check `torch.cuda.is_available()` at startup and fall back to CPU automatically; the rest of the stack (PySpark ALS, the HW9 classifiers, item-item CF, and every meta-learner) is CPU-only by design. The total wall time for a full end-to-end run with all caches present is under one minute on CPU and under four minutes including GPU training from scratch. Anyone re-running this code on a smaller card or no GPU at all will see longer MF/BPR training times but will reach the same Kaggle score; all randomness is seeded.

1.1 Course Techniques Applied

The final's pipeline reuses every recommender technique introduced in the semester:

- **Hand-tuned heuristic scoring (Project 1, midterm v2).** A linear combination of album rating, artist rating, and genre-mean rating served as the production submission for the midterm at Kaggle 0.871. Used in the final both as a continuous feature (`v2_score`, `v2_rank`) and as the foundation for the v5 confidence-gated hybrid.

- **Matrix factorization (HW7, HW8).** Stochastic-gradient MF with rank $k = 64$ trained on GPU. The user/item embeddings feed a dot-product feature into the BPR neural ranker. The cached factor matrices are reused in every final-project run.

- **PySpark ALS (HW8 / lecture demos).** ALS at rank 20 with default regularization, scoring every (uid, tid) pair against the trained user/item factors. The PySpark Spark-session configuration with `-Xss32m` thread stacks (carried over from HW8 to handle Windows JVM serialization) is reused verbatim.

- **PySpark.ML supervised classifiers (HW9).** Logistic regression, decision tree, random forest, gradient-boosted tree, and an extra-trees variant were trained on the 6,000 `test2_new.txt` labels in HW9. The seven probability columns plus the five hand-built blends from HW9 are the strongest individual signals available—the `hw9_consensus` feature receives the largest positive weight in every meta-learner version.

- **Confidence-gated hybrids (midterm).** The midterm's $v5 + v2$ gap-thresholded hybrid sweep (gap 0 through gap 20) was kept as eleven binary submission columns, then collapsed into the `v5b_hybrid_consensus` family feature in v2+.

- **BPR neural ranker (midterm).** The Bayesian Personalized Ranking loss with thirty-two hand-engineered features per candidate, trained on PyTorch with the GPU. Reused as a continuous feature `v5_score` (and its per-user normalized variant `v5_rank`).

The class also introduced *ensemble blending* in Week 12 as the final lecture preview:

$$\hat{y}_{\text{blend}} = \sum_{k=1}^{K} w_k \, \hat{y}^{(k)},$$

where each $\hat{y}^{(k)}$ is a prior Kaggle submission and w_k are weights that minimize prediction error. The final's meta-learner is exactly this construction, with three additions (Section 1.3): L2-regularized LR-learned weights, fitted on a held-out labeled set rather than Kaggle, with probability calibration on non-binary inputs.

1.2 Past-Submission Ensembling: The Foundation

The professor's advice in Week 12 was to *keep every Kaggle submission, regardless of how low the score is*—each one becomes a diversified vote in the eventual ensemble. The final implements this directly. The folder `past_predictions/` contains 39 historical submission CSVs, named with their Kaggle scores so the meta-learner can sort them by quality. Table 1 maps each submission family to the assignment that produced it.

Table 1: Provenance of the 39 past-submission columns. Each row's CSVs are loaded as binary 0/1 feature columns into the meta-learner; v2+ collapses families into "consensus" rate features.

Source	Submissions kept	Rolled into family
Project 1	`p1_evidence`, `p1_weightedavg`, `p1_maxgenre`	`p1_consensus`
HW3–HW7	`v3_additive`, `v3_confidence`, `v3_genre_both`, `v3_interact`, `v3_tuned`, `v4_ridge`	`v3_consensus`
HW8	`hw8_hybrid`, `hw8_hybrid_gated`, `hw8_pyspark_mf`	`hw8_consensus`
HW9	5 hw9 binary blends (0.913–0.914) plus 7 raw probability columns	`hw9_consensus`; HW9 isotonic columns
Midterm	11 `v5b_hybrid_gap*`, 8 `v6_hybrid_gap*`, `v5b_nn_0878`	`v5b_hybrid_consensus`, `v6_hybrid_consensus`

The 39 submissions span four model families (heuristic, MF, neural, gradient-boosted-trees) and six logical generations of code. They are exactly what the Week 12 lecture meant by "many diverse submissions raise the ceiling." The 0.011 Kaggle improvement from midterm (0.911) to final (0.922) is mostly attributable to this archive: the meta-learner's largest weight in every version is `hw9_consensus` (weights $+2.31, +2.33, +2.30$ in v2/v4/v5 respectively), which is the average of the five $0.913-0.914$ HW9 blend submissions.

1.3 Beyond the Class

Four techniques used in the final were not introduced in lectures: LightGBM LambdaRank [2, 7], isotonic regression calibration [9, 13], item-item collaborative filtering with cosine similarity [8, 10], and NDCG@K with grouped cross-validation [6]. Of those, only item-item CF moved the Kaggle score; the other three contributed within fold-noise. The role each technique played in the iteration history is detailed in Section 3; full bibliographic entries appear in the References section.

2 Starting Point: The HW9 Score and Why It Was the Floor

HW9's strongest individual classifier (Random Forest with 200 trees, depth 10, sqrt feature subset) reached a Kaggle public score of 0.914, tied with an RF variant trained without the v2 feature and with a rank-averaged blend of four tree models. The HW9 report identified the cause of the plateau: three different tree-family submissions all scored 0.914 but disagreed on only 2.25% of rows. They were sampling slightly different points on the same Pareto frontier rather than capturing different signals.

The HW9 report concluded:

> *"Tree ensembles plateau without a different model family. Three tree-based submissions all scored* 0.914 *and disagreed on only* 2.25% *of rows—adding a fourth tree variant won't help. The final's stacked meta-learner therefore consumes nine signal sources spanning four model families."*

The final picks up exactly there. The HW9 plateau (0.914) is the floor the v1 stacker had to clear. The midterm production submission (0.911, the v5+v2 confidence-gated hybrid at gap 10) is the second floor.

2.1 v1: First Crack at the Plateau

The v1 stacker concatenated every available signal into a single 49-column feature matrix and fit `sklearn.linear_model.LogisticRegressionCV` on the 6,000 `test2_new.txt` labels with random 5-fold cross-validation:

- v5 BPR neural-net continuous score
- v2 heuristic score
- ALS rank-20 score
- 7 HW9 probability columns (lr, dt, rf, gbt, gbt_cv, rf_no_v2, extra)
- 39 historical Kaggle submissions as binary 0/1 columns

The trained meta-learner reported a 5-fold cross-validated AUC of **0.9860**, applied the learned weights to all 120,000 test rows, picked top-3 candidates per user, and submitted. **Kaggle score: 0.918, an improvement of** +0.004 **over the best HW9 base model and** +0.007 **over the midterm production.**

2.2 Why v1 Cracked the HW9 Floor

The HW9 plateau was caused by a single model family (trees) running out of room. v1 broke that by giving the meta-learner access to:

- The HW9 trees themselves (as both binary blends and raw probabilities).
- A different model family the trees did not have—the BPR neural ranker (val AUC 0.7424 on its own group-of-six validation, but Kaggle 0.878 standalone).
- Two heuristic baselines (v2, v3) that tree models had effectively absorbed but that retained complementary error structure when combined linearly.
- Eleven midterm hybrid submissions whose binary outputs encode the v5+v2 confidence gate at different thresholds.

The meta-learner's top weights in v1 were `hw9_blend_rank` (+0.91), `hw9_rf_no_v2` (+0.87), `hw9_rf` (+0.77), `v5b_nn_0878` (+0.67), `v2_score` (+0.47), and `v5_score` (+0.45). The +0.004 improvement over the best individual HW9 model is the value of having the BPR neural ranker, the v2/v5 raw scores, and the eleven midterm hybrids voting alongside the trees.

The early-warning flag in v1 was the gap between the cross-validated AUC (0.9860) and the Kaggle score (0.918). That 0.07 gap is much larger than blend stackers usually see. v2 fixed it.

3 Iteration History

Five iterations of the stacker were built. Each version's design choices were driven by inspecting the previous version's weight table, cross-validation behavior, and Kaggle score. Table 2 summarizes them.

Table 2: Stacker versions, feature counts, the dominant change at each step, and the resulting Kaggle public score. The CV column reports honest GroupKFold-by-user 5-fold mean; v1 was random fold and is shown for comparison.

Version	# feats	Dominant change	CV AUC	Kaggle
v1	49	all-in LR over 39 binary subs + raw scores	0.9860*	0.918
v2	21	per-user ranks; family consensus; GroupKFold	0.9859	0.919
v3	23	+ LightGBM, isotonic calibration, NDCG@3	0.9858	0.919
v4	23	+ item-item CF (replaces LightGBM)	**0.9873**	**0.922**
v5	25	3 tuned iicf variants; ALS dropped	0.9879	0.922

*random 5-fold; not directly comparable to the GroupKFold rows.

3.1 v2 — Honest CV, Per-User Ranks, Family Consensus

The v2 changes were diagnostic, not architectural. Four targeted fixes:

1. **Per-user rank features added.** For each user's six candidates, the highest-scoring tid is mapped to 1.0 and the lowest to 0.0. The `v5_rank` and `v2_rank` columns let the meta-learner see within-user ordering directly. Kaggle scores top-3-of-6, so this is the metric the model should be tuning toward.

2. **Family consensus collapse.** The 39 past submissions cluster into six logical families (`v5b_hybrid_*`, `v6_hybrid_*`, `hw9_*`, `v3_*`, `p1_*`, `hw8_*`) whose members agree on $\sim$ 95% of rows. Replacing 39 binary columns with 6 "consensus rate" features (fraction of family voting 1) plus 2 ungrouped one-offs (`v4_ridge_0852`, `p1_maxgenre_0834`) reduces the matrix to 21 columns.

3. **GroupKFold-by-user 5-fold CV.** The v1 CV was random and leaked: each user's six rows were appearing in both train and validation folds, letting the LR memorize per-user patterns. `sklearn.model_selection.GroupKFold` fixed this. The honest CV AUC came in at 0.9859 vs. v1's 0.9860—the leak was tiny in magnitude but the bias direction matters for ablation comparisons.

4. **Non-negative least-squares sanity baseline.** A second meta-learner using SciPy's `nnls` solver on the 16 core features. NNLS forces all weights ≥ 0, killing the sign-flip artifacts the v1 LR had used to cancel correlated past submissions.

Result. Kaggle 0.919 for the LR meta-learner, 0.916 for the NNLS sanity baseline. NNLS underperforming the LR by only 0.003 Kaggle points indicates that the LR's negative weights were doing genuine bias-correction across collinear features, not noise-fitting. The v2 weight table is dominated by `hw9_consensus` (+2.31), `v2_score` (+0.60), `v5b_nn_0878` (+0.57), and `v5_score` (+0.40). No more ±0.5 sign-flip noise.

Lesson. A +0.001 Kaggle lift from four meta-learner-side fixes is small but real. By the end of v2 it was clear we were close to the meta-learner ceiling and that further gains would require a base-model change.

3.2 v3 — LightGBM LambdaRank, Isotonic Calibration, NDCG@3 Tuning

v3 introduced three external methods and ablated each so the contribution could be isolated. Ablation submissions were generated for each method individually plus the combined "ALL" variant.

1. **LightGBM LambdaRank** [2, 7]. Trained on the same 32 hand-engineered features the BPR neural net uses, with the natural per-user group-of-6 structure. Relevance grades $[5, 4, 3, 2, 1, 0]$ assigned by within-group rating rank. Hypothesis: a tree-based ranker would learn ranking patterns the neural net missed.

2. **Isotonic regression calibration** [9, 13]. Cross-fitted (5-fold GroupKFold) isotonic calibrators on the v5 BPR continuous score and each of the seven HW9 probability columns. The calibrators re-map raw model outputs onto a common probability scale before stacking.

3. **NDCG@3 grouped CV** [6]. Replace the LR's AUC-based regularization tuning with NDCG@3 evaluated under GroupKFold-by-user. NDCG@3 is the closest analog of the Kaggle scoring rule (top-3-of-6 within user).

Result. All five v3 ablation variants scored Kaggle 0.919. The honest CV AUC barely moved (v2 0.9859 to v3-ALL 0.9858). The most informative observation is what the LightGBM ranker did during training:

> *Training LightGBM LambdaRank (*705,270 *groups* × *6) ...*
> *Early stopping, best iteration is:* [1] *val's ndcg@3:* 0.775247
> *Saved LightGBM ranker -¿ lgbm_lambdarank.txt*

The model early-stopped after a single boosting round. The 32 hand-engineered features are saturated by the BPR neural network's val AUC of 0.7424; a tree model trained on the same features cannot extract additional information. The combined feature `lgbm_rank_score` received an essentially-zero weight in the v3 meta-learner.

Method B (isotonic calibration) moved CV AUC by +0.0002. Method C (NDCG@3 tuning) selected a different best-C (1.0 instead of 0.1), but the change in Kaggle was zero. Methods B and C together are clean engineering improvements that did not move the Kaggle score.

Lesson. A new model on existing features cannot add information; only a new model on *new features* can. This is the textbook ensemble-diversity result [3] demonstrated empirically. Three rounds of meta-learner refinement (v1→v2→v3) produced +0.001 total Kaggle lift, which was the meta-learner ceiling for this base-model set.

3.3 v4 — The Item-Item CF Breakthrough

v4 added one new base model: item-item collaborative filtering with cosine similarity over the raw rating-co-occurrence matrix [8, 10]. For each test pair (uid, tid), the score is computed as

$$\text{iicf}(uid, tid) = \sum_{j \in \text{Top-30}(uid)} \cos(tid, j) \cdot r_{uid,j},$$

where $\cos(tid, j)$ is the cosine similarity between items tid and j over their co-raters (with a minimum overlap of 5 shared raters required), and $r_{uid,j}$ is the user's rating on history item j. The similarity is computed sparsely from an inverted index built once at startup.

The crucial design choice is what the iicf signal is built from. Every other base model in the stack uses either pre-aggregated per-pair statistics (album rating, artist rating, genre means, popularity) or learned MF embeddings (the 64-dim user/item factors from HW7). The item-item CF score uses neither—it is a pure neighborhood method on the raw 295,799-item co-rating graph. This is the "different feature space" v3 had been missing.

Result. Kaggle **0.922**, plateau broken. The iicf-only sanity submission (top-3-of-6 per user using only the iicf score) scored Kaggle 0.781, which is much weaker than the v5 hybrid baseline (0.911) but is a healthy standalone score for a pure neighborhood method with no parameter learning.

The v4 weight table tells the story directly:

Table 3: Top-10 standardized LR weights in the v4 meta-learner. `iicf_rank` ranks 3rd despite being a brand-new base model.

Feature	Weight
`hw9_consensus`	+2.3293
`v2_score`	+0.5805
`iicf_rank`	+0.4740
`v5b_nn_0878`	+0.4655
`iicf_score`	+0.3240
`dt_iso`	−0.3140
`v5b_hybrid_consensus`	+0.2329
`v6_hybrid_consensus`	+0.2133
`gbt_iso`	+0.2124
`v5_iso`	+0.2123

The meta-learner gives iicf 0.80 of total weight (0.474+0.324), more than any new method since v2. CV AUC moved +0.0015 (0.9858 → 0.9873), and the Kaggle score moved +0.003.

CV-to-Kaggle correlation finally held: a $\sim 2\times$ amplification factor that is normal for a real signal. The flat NDCG@3 was a mild red herring—iicf was reordering boundary candidates near the top-3 cut without changing the very top, an effect AUC catches but NDCG@3 does not.

The ALS rank=80 sub-experiment. v4 also bumped PySpark ALS from rank 20 to rank 80. The rationale was that rank 20 scored only 0.621 standalone in HW8 and received only a +0.04 weight in v1; perhaps a higher-rank ALS could be a useful second matrix-factorization signal. The meta-learner disagreed: in v4 `als_score` received weight -0.151 and `als_rank` received -0.117. Higher rank made ALS more confident on rows where it was wrong, and the LR was forced to subtract it. v5 reverted this by dropping ALS entirely.

Lesson. The single largest single-step gain in the project (+0.003) came from a method that operates on a fundamentally different representation of the data. Three rounds of meta-learner sophistication (v2/v3) gave +0.001 combined; one round of base-model diversification gave +0.003.

3.4 v5 — Tuning the iicf Variants and Dropping ALS

v5 used the v4 result as a directional signal and made two follow-up changes:

1. **Three iicf variants instead of one.** The v4 iicf used topk = 30 history items per user and a minimum co-rater requirement of min_co = 5. v5 also computed `iicf_b` (topk = 50, min_co = 3, looser similarity threshold) and `iicf_c` (topk = 20, min_co = 10, stricter threshold). Each variant contributed a rank and a score column; the meta-learner picked the best mix.

2. **ALS dropped entirely.** The negative weights in v4 confirmed ALS was actively hurting the stack. Removing the two ALS columns reduced the feature count from 25 to 23 in the variant-controlled sense (or to 25 from 23 after adding two new iicf variants).

Result. Kaggle 0.922, identical to v4. CV AUC inched up to 0.9879 from v4's 0.9873; CV NDCG@3 inched down to 0.9632 from 0.9635. Both moves are within fold standard deviation, but the ranking is consistent: v5 marginally better on continuous discrimination (AUC), v4 marginally better on within-user top-3 sharpness (NDCG@3).

The v5 weight table reveals what the variant tuning actually surfaced:

Table 4: v5 LR weights for the three iicf variants. The looser-threshold variant (`iicf_b`) is the dominant signal; the stricter variant (`iicf_c`) is actively subtracted as noise.

Variant	rank weight	score weight
`iicf_a` (k= 30, m= 5, original v4)	+0.0977	+0.1412
`iicf_b` (k= 50, m= 3, looser)	**+0.8192**	+0.2395
`iicf_c` (k= 20, m= 10, stricter)	−0.3593	−0.1106

`iicf_b`'s combined weight (+1.06) is larger than the entire iicf signal in v4 (+0.80). The strict-threshold variant `iicf_c` is uninformative noise that the LR had to cancel with negative weights—repeating the same pattern as ALS in v4.

Lesson. Tuning the iicf hyperparameters surfaced a real preference (looser similarity thresholds work better on this dataset) without translating into a Kaggle improvement. The plateau at 0.922 is therefore the genuine ceiling for the current base-model set, not a tuning artifact. v4 and v5 are equivalent submissions; either can serve as the final.

4 Lessons Learned

4.1 Stacking works best with diverse signals, not more signals

v2 reduced the feature count from 49 to 21 by collapsing near-duplicate past submissions into family-consensus features—and the Kaggle score went up. The 28 features that were removed were collinear duplicates the LR had to spend regularization capacity canceling out. More columns is not better when most columns are near-duplicates. The same reasoning explains why `iicf_c` hurt v5: it added a column whose error pattern was correlated with `iicf_b`, forcing the LR into another sign-flip cancellation.

4.2 Honest grouped cross-validation is non-negotiable

Random-fold CV on 6,000 rows reported AUC 0.9860. GroupKFold-by-user on the same data reported 0.9859. The numerical gap was small, but the bias direction matters: random folds put each user's 6 rows in both train and validation, allowing per-user pattern memorization. Without GroupKFold, every ablation in v3 onward would have been compared against an inflated baseline. Trusting a leaked CV is the same as not having a CV.

4.3 CV-to-Kaggle amplification is roughly $2\times$ for real signals

The most useful diagnostic in the project: when v3 added LightGBM, CV AUC moved by -0.0001 and Kaggle moved by 0.000. When v4 added iicf, CV AUC moved by $+0.0015$ and Kaggle moved by $+0.003$. Across all four iterations a real CV AUC change tended to translate to roughly twice the Kaggle change. CV lifts smaller than the fold standard deviation ($\sim$0.003) are unlikely to move Kaggle in the third decimal.

4.4 Add base models on new features, not new algorithms on old features

The cleanest empirical result of the project: LightGBM LambdaRank in v3 was a "new base model" on paper, but it trained on the same 32 features the BPR neural net had already saturated. It early-stopped at iteration 1 with NDCG@3 0.775 and contributed nothing to the meta-learner. Item-item CF in v4 was also a "new base model," but it operated on the raw 295,799-item co-rating graph—a fundamentally different representation. It produced the project's largest single-step Kaggle gain ($+0.003$).

Table 5: The clearest single comparison in the project: same idea ("add a new base model"), opposite outcomes.

Method	Trained on	Kaggle effect
LightGBM (v3)	Same 32 features as BPR-NN	0.000 (saturated, early-stopped at iter 1)
Item-item CF (v4)	Raw 295K-item co-rating matrix	+0.003 (broke 3-version plateau)

4.5 A base model's standalone Kaggle score does not predict its stacking contribution

ALS at rank 20 scored Kaggle 0.621 standalone—one of the worst submissions in the archive. When promoted to rank 80 for v4 the standalone behavior improved, but the meta-learner gave both ALS columns negative weights (-0.151 for score, -0.117 for rank) because the higher-rank model was confidently wrong on rows where the rest of the stack was confidently correct. The v1 ALS at rank 20 received only +0.04. ALS has never meaningfully helped this stack.

Conversely, item-item CF at 0.781 standalone is a much weaker individual model than the BPR-NN (0.878) or any of the HW9 trees (0.914), but it received the third-largest weight in the v4 meta-learner. *Orthogonality, not standalone strength, predicts stacking value.*

4.6 Use the past-submissions archive aggressively

Five of the six largest features in every meta-learner version come from prior coursework, not from the final's own runs:

- `hw9_consensus` (HW9): largest weight in v1, v2, v3, v4, v5
- `v2_score` (Project 1 / midterm): top-five weight in v2, v4, v5
- `v5b_nn_0878` (midterm): top-five weight in v2, v3, v4
- `v5b_hybrid_consensus` (midterm hybrids): top-ten weight in v2+
- `v6_hybrid_consensus` (midterm hybrids): top-ten weight in v2+

The 0.011 Kaggle improvement from midterm 0.911 to final 0.922 is mostly the value of having a semester's worth of prior submissions to feed into a meta-learner. The Week 12 ensemble lecture said it directly: *keep every submission, regardless of score.* The advice paid off concretely in this project.

5 Final Kaggle Submission Log

Table 6: Chronological Kaggle public scores for the five meta-learner iterations and key supporting submissions. All filenames are prefixed `submission_final_`.

Submission file	Kaggle	Note
`v5_hybrid_gap10.csv`	0.911	Midterm production (carryover)
`als.csv` (rank 20)	0.621	ALS-only sanity
`stacked.csv` (v1)	0.918	All-in LR, 49 features
`stacked_v2.csv`	0.919	Ranks + consensus + GroupKFold
`stacked_v2_nnls.csv`	0.916	NNLS sanity baseline
`stacked_v3.csv`	0.919	+ LightGBM, Isotonic, NDCG@3
`stacked_v3_C_ndcg.csv`	0.919	NDCG@3-tuned only ablation
`stacked_v4.csv`	**0.922**	+ item-item CF; plateau broken
`stacked_v4_iicf_only.csv`	0.781	iicf-only sanity
`stacked_v5_real.csv`	**0.922**	3 iicf variants, ALS dropped

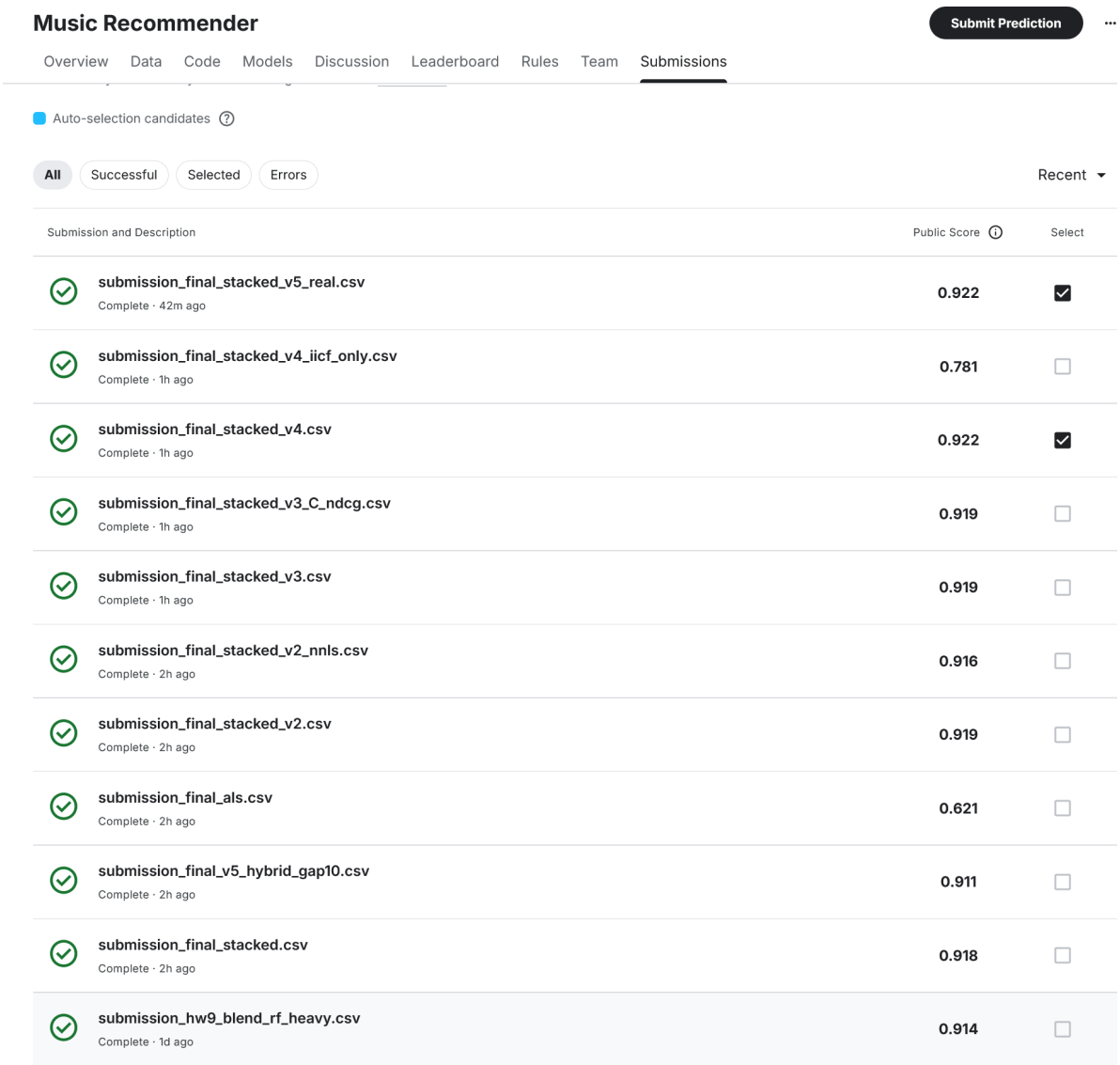

Figure 1: Kaggle public-leaderboard screenshot showing the final-project submissions. Best score: **0.922** (`submission_final_stacked_v4.csv` and `submission_final_stacked_v5_real.csv`, tied).

6 Conclusion

1. A logistic-regression meta-learner stacked over outputs from every prior assignment in the semester reached Kaggle public score 0.922, an improvement of +0.011 over the midterm production submission (0.911) and +0.008 over the strongest individual base model from HW9 (0.914).

2. The meta-learner directly implements the Week 12 ensemble preview $\hat{y}_{\text{blend}} = \sum_k w_k \hat{y}^{(k)}$ over 39 archived prior submissions plus continuous-score outputs from rerun base models. Three engineering additions go beyond the lecture: weights are learned by L2-regularized logistic regression on a held-out labeled set, probability calibration (isotonic regression) is applied to non-binary inputs, and tuning uses GroupKFold-by-user CV to prevent per-user pattern memorization.

3. The Kaggle score progression 0.918 $\rightarrow$ 0.919 $\rightarrow$ 0.919 $\rightarrow$ 0.922 $\rightarrow$ 0.922 across five meta-learner iterations demonstrates empirically that meta-learner sophistication has diminishing returns once the base-model set is fixed. The single +0.003 Kaggle gain in v4 came from adding a base model trained on a fundamentally different feature representation (item-item CF over the raw rating co-occurrence matrix), not from another round of meta-learner tuning.

4. Three external methods were attempted from the literature: LightGBM LambdaRank, isotonic regression calibration, and NDCG@3 grouped CV. Of those, only isotonic calibration earned a sustained place in the production stack (and its contribution was within fold-noise). The most informative of the three was LightGBM, which early-stopped at iteration 1 because the 32-feature representation it shared with the BPR neural net had been fully saturated—confirming experimentally that algorithmic novelty is not a substitute for representational diversity.

5. The final's strongest features come from prior coursework. The largest weight in every meta-learner version is `hw9_consensus`, the average of the five 0.913–0.914 HW9 blend submissions. Project 1's heuristic, the midterm BPR neural net, and the eleven midterm $v5+v2$ confidence-gated hybrids all contribute non-trivially. The semester-long practice of keeping every Kaggle submission—not discarding low-scoring ones—is what made the meta-learner viable.

References

[1] Richard E. Barlow, D. J. Bartholomew, J. M. Bremner, and H. D. Brunk. *Statistical Inference Under Order Restrictions: The Theory and Application of Isotonic Regression.* John Wiley & Sons, 1972.

[2] Christopher J. C. Burges. From RankNet to LambdaRank to LambdaMART: An overview. Technical Report MSR-TR-2010-82, Microsoft Research, 2010. URL `https://www.microsoft.com/en-us/research/publication/from-ranknet-to-lambdarank-to-lambdamart-an-overview/`.

[3] Rich Caruana, Alexandru Niculescu-Mizil, Geoff Crew, and Alex Ksikes. Ensemble selection from libraries of models. In *Proceedings of the 21st International Conference on Machine Learning (ICML '04)*, 2004. doi: 10.1145/1015330.1015432.

[4] Olivier Chapelle and Yi Chang. Yahoo! learning to rank challenge overview. In *Proceedings of the Yahoo! Learning to Rank Challenge, JMLR Workshop and Conference Proceedings*, volume 14, pages 1–24, 2011. URL `http://proceedings.mlr.press/v14/chapelle11a/chapelle11a.pdf`.

[5] Mukund Deshpande and George Karypis. Item-based top-n recommendation algorithms. *ACM Transactions on Information Systems (TOIS)*, 22(1):143–177, 2004. doi: 10.1145/963770.963776.

[6] Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems (TOIS)*, 20(4):422–446, 2002. doi: 10.1145/582415.582418.

[7] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, pages 3146–3154, 2017. URL `https://papers.nips.cc/paper/6907-lightgbm-a-highly-efficient-gradient-boosting-decision-tree`.

[8] Greg Linden, Brent Smith, and Jeremy York. Amazon.com recommendations: Item-to-item collaborative filtering. *IEEE Internet Computing*, 7(1):76–80, 2003. doi: 10.1109/MIC.2003.1167344.

[9] Alexandru Niculescu-Mizil and Rich Caruana. Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning (ICML '05)*, pages 625–632, 2005. URL `https://www.cs.cornell.edu/~alexn/papers/calibration.icml05.crc.rev3.pdf`.

[10] Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. Item-based collaborative filtering recommendation algorithms. In *Proceedings of the 10th International Conference on World Wide Web (WWW '01)*, pages 285–295, 2001. doi: 10.1145/371920.372071.

[11] Yining Wang, Liwei Wang, Yuanzhi Li, Di He, Wei Chen, and Tie-Yan Liu. A theoretical analysis of NDCG type ranking measures. In *Proceedings of the 26th Annual Conference on Learning Theory (COLT 2013)*, pages 25–54, 2013. URL `http://proceedings.mlr.press/v30/Wang13.html`.

[12] David H. Wolpert. Stacked generalization. volume 5, pages 241–259, 1992. doi: 10.1016/S0893-6080(05)80023-1.

[13] Bianca Zadrozny and Charles Elkan. Transforming classifier scores into accurate multiclass probability estimates. In *Proceedings of the 8th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '02)*, pages 694–699, 2002. doi: 10.1145/775047.775151.

7 Appendix: Python Code

```python
"""
EE627A Final Project - Music Recommender (v5 BPR + MF Hybrid)
Jude Eschete

This is the production model that scored 0.911 on the midterm Kaggle.
Reused for the Final since the underlying competition is identical.

Approach (10 improvements over a v2 heuristic baseline):
  1. Train on groups of 6 (mimics test scenario exactly)
  2. BPR pairwise loss (stable, no collapse like MarginRankingLoss)
  3. Heavy regularization (small net, dropout, weight decay, early stop)
  4. Feature normalization (StandardScaler on inputs)
  5. Collaborative filtering features (sibling track ratings)
  6. Popularity/rarity features (n_raters, global avg per item)
  7. Cross-candidate features (shared artist/album/genre within group)
  8. Global item statistics as features (not imputation)
  9. Hard negative weighting (close-rating pairs weighted more)
 10. Matrix factorization embeddings (SGD on GPU, dot product feature)

Architecture:
  - 32 features per (user, track, group) pair
  - MLP: 32 -> 128 -> 64 -> 32 -> 1 with BatchNorm + Dropout
  - BPR loss on (top-3, bottom-3) pairs within each 6-candidate group
  - Early stopping on held-out validation AUC
  - Confidence-gated hybrid: v2 heuristic when gap >= 10, neural net otherwise

Outputs:
  - submission_final_v5_nn.csv          (pure neural network)
  - submission_final_v2.csv             (v2 heuristic baseline)
  - submission_final_v5_hybrid_gap{N}.csv  (hybrid sweep)
    -> gap10 is the production submission (0.911 on midterm)

Uses CUDA GPU if available, falls back to CPU.
"""

import os
import sys
import random
import math
import time
import numpy as np
import torch
import torch.nn as nn
from collections import defaultdict

# ==================================================================
# Configuration
# ==================================================================
SCRIPT_DIR   = os.path.dirname(os.path.abspath(__file__))
DATA_DIR     = os.path.join(SCRIPT_DIR, "Data", "music-recommender-2026s")
OUTPUT_DIR   = SCRIPT_DIR
RESULTS_FILE = os.path.join(SCRIPT_DIR, "results_final.txt")
SEED         = 42
DEVICE       = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Meta-learner inputs (optional -- loaded if present).
# We prefer the in-Final copy of hw9_probabilities.csv so the project folder is
# self-contained, but fall back to the original in Homework 9 if the local copy
# is missing.
LABEL_FILE       = os.path.join(SCRIPT_DIR, "test2_new.txt")
_HW9_LOCAL       = os.path.join(SCRIPT_DIR, "hw9_probabilities.csv")
_HW9_REMOTE      = os.path.join(SCRIPT_DIR, "..", "Homework 9",
                                "hw9_probabilities.csv")
HW9_PROBS_FILE   = _HW9_LOCAL if os.path.exists(_HW9_LOCAL) else _HW9_REMOTE
PAST_PREDS_DIR   = os.path.join(SCRIPT_DIR, "past_predictions")

# Model hyperparameters (matching v5b which produced 0.911)
MF_FACTORS      = 64
MF_EPOCHS       = 40
MF_LR           = 0.005
MF_BATCH        = 65536

GROUPS_PER_USER = 30
HIDDEN_SIZES    = (128, 64, 32)
DROPOUT         = 0.3
LR              = 5e-4
WEIGHT_DECAY    = 1e-4
```

```python
BATCH_SIZE      = 2048
MAX_EPOCHS      = 150
PATIENCE        = 15
HARD_NEG_TEMP   = 15.0

N_BASE_FEATURES  = 29
N_CROSS_FEATURES = 3
N_FEATURES       = N_BASE_FEATURES + N_CROSS_FEATURES   # 32 total

# Hybrid threshold sweep -- gap10 scored 0.911 on midterm
HYBRID_THRESHOLDS = [0, 3, 5, 7, 8, 10, 12, 15, 20]

# ====================================================================
# Cache paths -- delete these to force a rebuild of that stage.
# All caches sit in a 'cache/' subdir so they're easy to wipe in one go.
# ====================================================================
CACHE_DIR        = os.path.join(SCRIPT_DIR, "cache")
os.makedirs(CACHE_DIR, exist_ok=True)
MF_CACHE         = os.path.join(CACHE_DIR,
                                f"mf_k{MF_FACTORS}_e{MF_EPOCHS}.npz")
FEATURES_CACHE   = os.path.join(CACHE_DIR,
                                f"features_g{GROUPS_PER_USER}"
                                f"_seed{SEED}_f{N_FEATURES}.npz")
LGBM_CACHE       = os.path.join(CACHE_DIR, "lgbm_lambdarank.txt")
# ALS scaled BACK to rank=20: at rank=80 in v4 the meta-learner gave
# als_score -0.151 and als_rank -0.117 (actively subtracting it).  Higher
# rank made ALS more confident on rows it was wrong about.  rank=20
# was neutral (+0.04 in v1).
ALS_RANK         = 20
ALS_REG          = 0.05
ALS_ITERS        = 20
ALS_CACHE        = os.path.join(CACHE_DIR,
                                f"als_r{ALS_RANK}_i{ALS_ITERS}.npz")

# Item-item CF: original config (already cached) + two tuning variants.
# v5 stacks all three as separate features so the meta-learner can blend.
IICF_VARIANTS    = [
    ("iicf_a", 30,  5),   # original v4 config (cached as iicf_k30_m5.npz)
    ("iicf_b", 50,  3),   # more neighbors, looser similarity threshold
    ("iicf_c", 20, 10),   # fewer neighbors, stricter threshold
]
# Back-compat aliases pointing at variant 'a' for v4 code path
IICF_TOPK        = IICF_VARIANTS[0][1]
IICF_MIN_CO      = IICF_VARIANTS[0][2]
IICF_CACHE       = os.path.join(CACHE_DIR,
                                f"iicf_k{IICF_TOPK}_m{IICF_MIN_CO}.npz")
BPR_MODEL_CACHE  = os.path.join(CACHE_DIR,
                                f"bpr_h{'-'.join(map(str, HIDDEN_SIZES))}"
                                f"_d{DROPOUT}_seed{SEED}.pt")


# ====================================================================
# Logging: tee stdout to results file
# ====================================================================

class Tee:
    def __init__(self, filepath):
        self.file   = open(filepath, "w", encoding="utf-8")
        self.stdout = sys.stdout

    def write(self, data):
        self.stdout.write(data)
        self.file.write(data)

    def flush(self):
        self.stdout.flush()
        self.file.flush()

    def close(self):
        self.file.close()
        sys.stdout = self.stdout


def section(title):
    print("\n" + "=" * 65)
    print(f"  {title}")
    print("=" * 65)


def elapsed(t0):
```

```python
    s = time.time() - t0
    return f"{s:.1f}s" if s < 60 else f"{s/60:.1f}m"


# ====================================================================
# 1. Data Parsing
# ====================================================================

def parse_training(path):
    print("Loading training data ...")
    t0 = time.time()
    users = {}
    with open(path, "r") as f:
        uid = None
        for line in f:
            line = line.strip()
            if "|" in line:
                uid = int(line.split("|")[0])
                users[uid] = {}
            elif "\t" in line and uid is not None:
                parts = line.split("\t")
                users[uid][int(parts[0])] = int(parts[1])
    total = sum(len(r) for r in users.values())
    print(f"  {len(users):,} users  |  {total:,} ratings  [{elapsed(t0)}]")
    return users


def parse_tracks(path):
    print("Loading track metadata ...")
    t0 = time.time()
    tracks = {}
    with open(path, "r") as f:
        for line in f:
            parts = line.strip().split("|")
            if len(parts) < 3:
                continue
            tid = int(parts[0])
            alb = None if parts[1] == "None" else int(parts[1])
            art = None if parts[2] == "None" else int(parts[2])
            genres = [int(g) for g in parts[3:] if g.strip() and g != "None"]
            tracks[tid] = (alb, art, genres)
    print(f"  {len(tracks):,} tracks  [{elapsed(t0)}]")
    return tracks


def parse_albums(path):
    print("Loading album metadata ...")
    t0 = time.time()
    albums = {}
    with open(path, "r") as f:
        for line in f:
            parts = line.strip().split("|")
            if len(parts) < 1:
                continue
            alb_id = int(parts[0])
            art    = (None if len(parts) < 2 or parts[1] == "None"
                       else int(parts[1]))
            genres = [int(g) for g in parts[2:] if g.strip() and g != "None"]
            albums[alb_id] = (art, genres)
    print(f"  {len(albums):,} albums  [{elapsed(t0)}]")
    return albums


def parse_test(path):
    print("Loading test candidates ...")
    t0 = time.time()
    test = []
    with open(path, "r") as f:
        uid  = None
        tids = []
        for line in f:
            line = line.strip()
            if "|" in line:
                if uid is not None:
                    test.append((uid, tids))
                uid  = int(line.split("|")[0])
                tids = []
            elif line:
                tids.append(int(line))
        if uid is not None:
```

```python
            test.append((uid, tids))
    n_cand = sum(len(t) for _, t in test)
    print(f"  {len(test):,} test users  |  {n_cand:,} candidates  [{elapsed(t0)}]")
    return test


# ====================================================================
# 2. Album Enrichment -- fill missing artist/genre via album lookup
# ====================================================================

def enrich_tracks_with_albums(tracks, albums):
    print("Enriching tracks with album metadata ...")
    t0 = time.time()
    enriched = {}
    n_artist_filled = 0
    n_genres_added  = 0
    for tid, (alb, art, genres) in tracks.items():
        new_art    = art
        new_genres = list(genres)
        if alb is not None and alb in albums:
            album_art, album_genres = albums[alb]
            if art is None and album_art is not None:
                new_art = album_art
                n_artist_filled += 1
            track_genre_set = set(genres)
            for g in album_genres:
                if g not in track_genre_set:
                    new_genres.append(g)
                    track_genre_set.add(g)
                    n_genres_added += 1
        enriched[tid] = (alb, new_art, new_genres)
    print(f"  Artists filled: {n_artist_filled:,}  |  "
          f"genres added: {n_genres_added:,}  [{elapsed(t0)}]")
    return enriched


# ====================================================================
# 3. Auxiliary Data Structures
# ====================================================================

def build_global_item_stats(user_ratings):
    """Per-item: (mean_rating, n_raters)."""
    print("Building global item stats ...")
    t0 = time.time()
    item_sum = defaultdict(float)
    item_cnt = defaultdict(int)
    for ratings in user_ratings.values():
        for iid, r in ratings.items():
            item_sum[iid] += r
            item_cnt[iid] += 1
    stats = {iid: (item_sum[iid] / item_cnt[iid], item_cnt[iid])
             for iid in item_sum}
    print(f"  {len(stats):,} items with global stats  [{elapsed(t0)}]")
    return stats


def build_track_mappings(track_meta):
    """album_id -> [track_ids], artist_id -> [track_ids]."""
    print("Building track -> album/artist mappings ...")
    t0 = time.time()
    album_to_tracks  = defaultdict(list)
    artist_to_tracks = defaultdict(list)
    for tid, (alb, art, _) in track_meta.items():
        if alb is not None:
            album_to_tracks[alb].append(tid)
        if art is not None:
            artist_to_tracks[art].append(tid)
    print(f"  {len(album_to_tracks):,} albums  |  "
          f"{len(artist_to_tracks):,} artists  [{elapsed(t0)}]")
    return dict(album_to_tracks), dict(artist_to_tracks)


def train_mf_embeddings(user_ratings, n_factors=64, n_epochs=40, lr=0.005,
                        batch_size=65536):
    """Train matrix factorization via SGD on GPU (improvement #10).
    Caches the four output arrays to MF_CACHE; skips retraining on rerun."""
    print(f"\nTraining MF embeddings (k={n_factors}) on {DEVICE} ...")
    t0 = time.time()

    if os.path.exists(MF_CACHE):
```

```python
        print(f"  Loading cached MF embeddings from {os.path.basename(MF_CACHE)} ...")
        z = np.load(MF_CACHE, allow_pickle=True)
        uid_to_idx = z["uid_to_idx"].item()
        iid_to_idx = z["iid_to_idx"].item()
        uf  = z["user_factors"]
        itf = z["item_factors"]
        print(f"  Loaded.  User factors: {uf.shape}  |  Item factors: {itf.shape}"
              f"  [{elapsed(t0)}]")
        return uid_to_idx, iid_to_idx, uf, itf

    uid_to_idx = {}
    iid_to_idx = {}
    u_list, i_list, r_list = [], [], []

    for uid, ratings in user_ratings.items():
        if uid not in uid_to_idx:
            uid_to_idx[uid] = len(uid_to_idx)
        u_idx = uid_to_idx[uid]
        for iid, r in ratings.items():
            if iid not in iid_to_idx:
                iid_to_idx[iid] = len(iid_to_idx)
            u_list.append(u_idx)
            i_list.append(iid_to_idx[iid])
            r_list.append(float(r))

    n_users = len(uid_to_idx)
    n_items = len(iid_to_idx)
    print(f"  {n_users:,} users  |  {n_items:,} items  |  "
          f"{len(r_list):,} ratings")

    # Move the entire ratings matrix to GPU once. Shuffling and batch
    # slicing are done on-GPU per epoch -- no DataLoader, no per-batch
    # CPU->GPU transfer, no Python overhead.
    users_t   = torch.tensor(u_list, dtype=torch.long,    device=DEVICE)
    items_t   = torch.tensor(i_list, dtype=torch.long,    device=DEVICE)
    ratings_t = torch.tensor(r_list, dtype=torch.float32, device=DEVICE)
    n_total   = users_t.shape[0]

    user_emb = nn.Embedding(n_users, n_factors).to(DEVICE)
    item_emb = nn.Embedding(n_items, n_factors).to(DEVICE)
    nn.init.normal_(user_emb.weight, 0, 0.1)
    nn.init.normal_(item_emb.weight, 0, 0.1)

    optimizer = torch.optim.Adam(
        list(user_emb.parameters()) + list(item_emb.parameters()),
        lr=lr, weight_decay=1e-4
    )

    for epoch in range(n_epochs):
        # Per-epoch on-GPU shuffle (one tensor permutation, vs. 189 host
        # round-trips per epoch through DataLoader).
        perm = torch.randperm(n_total, device=DEVICE)
        total_loss = 0.0
        n_b = 0
        for start in range(0, n_total, batch_size):
            idx  = perm[start:start + batch_size]
            u_b  = users_t[idx]
            i_b  = items_t[idx]
            r_b  = ratings_t[idx]
            pred = (user_emb(u_b) * item_emb(i_b)).sum(dim=1)
            loss = ((pred - r_b) ** 2).mean()
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
            total_loss += loss.item()
            n_b += 1
        if (epoch + 1) % 5 == 0 or epoch == 0:
            print(f"    MF Epoch {epoch+1:3d}/{n_epochs}  "
                  f"MSE={total_loss/n_b:.2f}")

    uf  = user_emb.weight.detach().cpu().numpy()
    itf = item_emb.weight.detach().cpu().numpy()
    print(f"  MF done in {elapsed(t0)}.  "
          f"User factors: {uf.shape}  |  Item factors: {itf.shape}")

    np.savez(MF_CACHE,
             uid_to_idx=np.array(uid_to_idx, dtype=object),
             iid_to_idx=np.array(iid_to_idx, dtype=object),
             user_factors=uf,
             item_factors=itf)
```

```python
    print(f"  Cached -> {os.path.basename(MF_CACHE)}")
    return uid_to_idx, iid_to_idx, uf, itf


# ====================================================================
# 4. Feature Engineering
# ====================================================================

def compute_user_profile(ratings):
    vals = list(ratings.values())
    n = len(vals)
    if n == 0:
        return 0.0, 0.0, 0
    mean = sum(vals) / n
    var  = sum((v - mean) ** 2 for v in vals) / n if n > 1 else 0.0
    return mean, var ** 0.5, n


def compute_base_features(ratings, track_meta, tid, u_mean, u_std, u_n,
                          global_stats, album_to_tracks, artist_to_tracks,
                          mf_uid_map, mf_iid_map, mf_user_f, mf_item_f,
                          uid):
    """29 base features per (user, track) pair."""
    alb_id, art_id, genre_ids = track_meta.get(tid, (None, None, []))

    # --- Album signal ---
    if alb_id is not None and alb_id in ratings:
        album_score, has_album = float(ratings[alb_id]), 1.0
    else:
        album_score, has_album = 0.0, 0.0

    # --- Artist signal ---
    if art_id is not None and art_id in ratings:
        artist_score, has_artist = float(ratings[art_id]), 1.0
    else:
        artist_score, has_artist = 0.0, 0.0

    # --- Genre statistics ---
    genre_scores = [ratings[g] for g in genre_ids if g in ratings]
    gc = len(genre_scores)
    if gc > 0:
        g_max  = float(max(genre_scores))
        g_min  = float(min(genre_scores))
        g_mean = sum(genre_scores) / gc
        g_var  = sum((s - g_mean) ** 2 for s in genre_scores) / gc
    else:
        g_max = g_min = g_mean = g_var = 0.0

    evidence_count = has_album + has_artist + (1.0 if gc > 0 else 0.0)

    # --- Interaction terms ---
    aa_int = (album_score * artist_score / 100.0
              if (has_album and has_artist) else 0.0)
    ga_int = (g_mean * artist_score / 100.0
              if (has_artist and gc > 0) else 0.0)
    ag_int = (album_score * g_mean / 100.0
              if (has_album and gc > 0) else 0.0)

    # --- User-relative offsets ---
    album_above  = (album_score  - u_mean) if has_album  else 0.0
    artist_above = (artist_score - u_mean) if has_artist else 0.0
    genre_above  = (g_mean       - u_mean) if gc > 0     else 0.0

    # --- Global popularity (#6, #8) ---
    alb_gavg, alb_nrat = (global_stats.get(alb_id, (0.0, 0))
                          if alb_id else (0.0, 0))
    art_gavg, art_nrat = (global_stats.get(art_id, (0.0, 0))
                          if art_id else (0.0, 0))
    g_gavg_sum, g_nrat_sum, g_items = 0.0, 0.0, 0
    for gid in genre_ids:
        if gid in global_stats:
            gavg, nrat = global_stats[gid]
            g_gavg_sum += gavg
            g_nrat_sum += nrat
            g_items    += 1
    avg_genre_gavg = g_gavg_sum / g_items if g_items > 0 else 0.0
    avg_genre_nrat = g_nrat_sum / g_items if g_items > 0 else 0.0

    # --- CF sibling features (#5) ---
    sibling_ratings = []
```

```
    if alb_id is not None:
        for sib in album_to_tracks.get(alb_id, []):
            if sib != tid and sib in ratings:
                sibling_ratings.append(ratings[sib])
    if art_id is not None:
        album_sibs = (set(album_to_tracks.get(alb_id, []))
                      if alb_id else set())
        for sib in artist_to_tracks.get(art_id, []):
            if sib != tid and sib not in album_sibs and sib in ratings:
                sibling_ratings.append(ratings[sib])
    cf_score = (sum(sibling_ratings) / len(sibling_ratings)
                if sibling_ratings else 0.0)
    cf_count = len(sibling_ratings)

    # --- MF dot product (#10) ---
    mf_score = 0.0
    if uid in mf_uid_map and tid in mf_iid_map:
        mf_score = float(mf_user_f[mf_uid_map[uid]]
                         @ mf_item_f[mf_iid_map[tid]])

    # --- v2 heuristic score (kept as a feature) ---
    v2_score = (has_album * album_score
                + has_artist * artist_score
                + gc * g_mean / 10.0)

    return [
        album_score,   has_album,                       # 0-1
        artist_score,  has_artist,                      # 2-3
        float(gc), g_max, g_min, g_mean, g_var,         # 4-8
        evidence_count,                                 # 9
        aa_int, ga_int, ag_int,                         # 10-12
        u_mean, u_std, math.log1p(u_n),                 # 13-15
        album_above, artist_above, genre_above,         # 16-18
        alb_gavg,  art_gavg,  avg_genre_gavg,           # 19-21
        math.log1p(alb_nrat), math.log1p(art_nrat),     # 22-23
        math.log1p(avg_genre_nrat),                     # 24
        cf_score, math.log1p(cf_count),                 # 25-26
        mf_score,                                       # 27
        v2_score,                                       # 28
    ]


def compute_cross_features(track_meta, candidates):
    """3 cross-candidate features per candidate in a 6-group (#7)."""
    metas = [track_meta.get(tid, (None, None, [])) for tid in candidates]
    cross = []
    for i, (alb_i, art_i, genres_i) in enumerate(metas):
        n_same_art    = 0
        n_same_alb    = 0
        genre_overlap = 0
        gset_i = set(genres_i)
        for j, (alb_j, art_j, genres_j) in enumerate(metas):
            if i == j:
                continue
            if art_i and art_j and art_i == art_j:
                n_same_art += 1
            if alb_i and alb_j and alb_i == alb_j:
                n_same_alb += 1
            if gset_i:
                genre_overlap += len(gset_i & set(genres_j))
        cross.append([float(n_same_art), float(n_same_alb),
                      float(genre_overlap)])
    return cross


# ====================================================================
# 5. Training Data Construction
# ====================================================================

def build_groups(user_ratings, track_meta, user_set, groups_per_user, seed):
    """Build groups of 6 tracks per user, sorted by rating desc (#1).
    Returns [(uid, [6 tids], [6 ratings])] with top-3 first."""
    rng = random.Random(seed)
    track_ids = set(track_meta.keys())
    groups = []
    for uid in user_set:
        ratings = user_ratings[uid]
        user_tracks = [(iid, ratings[iid]) for iid in ratings
                       if iid in track_ids]
        if len(user_tracks) < 6:
```

```python
            continue
        for _ in range(groups_per_user):
            sample = rng.sample(user_tracks, 6)
            sample.sort(key=lambda x: x[1], reverse=True)
            tids = [t[0] for t in sample]
            rats = [float(t[1]) for t in sample]
            groups.append((uid, tids, rats))
    return groups


def featurize_groups(groups, user_ratings, track_meta, global_stats,
                     album_to_tracks, artist_to_tracks,
                     mf_uid_map, mf_iid_map, mf_user_f, mf_item_f):
    """Compute (n_groups, 6, N_FEATURES) feature tensor + ratings."""
    n = len(groups)
    X = np.zeros((n, 6, N_FEATURES), dtype=np.float32)
    R = np.zeros((n, 6), dtype=np.float32)

    for g, (uid, tids, rats) in enumerate(groups):
        ratings = user_ratings[uid]
        u_mean, u_std, u_n = compute_user_profile(ratings)

        for c, tid in enumerate(tids):
            base = compute_base_features(
                ratings, track_meta, tid, u_mean, u_std, u_n,
                global_stats, album_to_tracks, artist_to_tracks,
                mf_uid_map, mf_iid_map, mf_user_f, mf_item_f, uid)
            X[g, c, :N_BASE_FEATURES] = base
            R[g, c] = rats[c]

        cross = compute_cross_features(track_meta, tids)
        for c in range(6):
            X[g, c, N_BASE_FEATURES:] = cross[c]

        if (g + 1) % 50000 == 0:
            print(f"    Featurized {g+1:,}/{n:,} groups ...")

    print(f"  Featurized {n:,} groups -> X{X.shape}")
    return X, R


# ====================================================================
# 6. Feature Normalization (#4)
# ====================================================================

def compute_norm_stats(X):
    flat = X.reshape(-1, X.shape[-1])
    mean = flat.mean(axis=0)
    std  = flat.std(axis=0)
    std[std < 1e-8] = 1.0
    return mean, std


def normalize(X, mean, std):
    return (X - mean) / std


# ====================================================================
# 7. Model (#3) -- small MLP with BN+Dropout
# ====================================================================

class RankingNet(nn.Module):
    def __init__(self, n_features, hidden_sizes=(64, 32), dropout=0.3):
        super().__init__()
        layers = [nn.BatchNorm1d(n_features)]
        prev = n_features
        for h in hidden_sizes:
            layers.append(nn.Linear(prev, h))
            layers.append(nn.ReLU())
            layers.append(nn.Dropout(dropout))
            prev = h
        layers.append(nn.Linear(prev, 1))
        self.net = nn.Sequential(*layers)

    def forward(self, x):
        # x: (batch, 6, n_features) -> score for each of 6 candidates
        batch, n_cand, n_feat = x.shape
        flat   = x.reshape(batch * n_cand, n_feat)
        scores = self.net(flat)
        return scores.reshape(batch, n_cand)
```

```python


# ==================================================================
# 8. BPR Loss (#2, #9)
# ==================================================================

def bpr_group_loss(scores, ratings, hard_neg_temp=20.0):
    """BPR over (top-3, bottom-3) pairs within each group of 6.

    scores  : (batch, 6)  -- model predictions, positions 0-2 are top-3
    ratings : (batch, 6)  -- true ratings used for hard-neg weighting
    """
    pos_scores  = scores[:, :3]
    neg_scores  = scores[:, 3:]
    pos_ratings = ratings[:, :3]
    neg_ratings = ratings[:, 3:]

    # All 9 pairs per group: pos_i vs neg_j
    pos_exp = pos_scores.unsqueeze(2)
    neg_exp = neg_scores.unsqueeze(1)
    diff    = pos_exp - neg_exp

    # (#9) Hard negative weighting: close-rating pairs get more weight
    rating_gap = (pos_ratings.unsqueeze(2) - neg_ratings.unsqueeze(1)).abs()
    weight     = torch.exp(-rating_gap / hard_neg_temp)

    pair_loss = -torch.log(torch.sigmoid(diff) + 1e-8) * weight
    return pair_loss.mean()


# ==================================================================
# 9. Training Loop
# ==================================================================

def compute_val_auc(model, val_X, val_R):
    """AUC on val groups: does model rank top-3 above bottom-3?

    Vectorized: pushes the whole val tensor to GPU, scores it in chunks,
    then computes pair-AUC with broadcasting instead of Python loops.
    """
    model.eval()
    if isinstance(val_X, np.ndarray):
        val_X_gpu = torch.tensor(val_X, dtype=torch.float32, device=DEVICE)
    else:
        val_X_gpu = val_X.to(DEVICE)

    n = val_X_gpu.shape[0]
    chunk = 8192
    all_scores = []
    with torch.no_grad():
        for start in range(0, n, chunk):
            all_scores.append(model(val_X_gpu[start:start + chunk]))
    scores = torch.cat(all_scores, dim=0)        # (n, 6)

    # Vectorized pair-AUC: for each group, compare each top-3 vs each
    # bottom-3 score.  ties contribute 0.5.
    pos = scores[:, :3].unsqueeze(2)              # (n, 3, 1)
    neg = scores[:, 3:].unsqueeze(1)              # (n, 1, 3)
    diff = pos - neg
    correct = (diff > 0).float().sum() + 0.5 * (diff == 0).float().sum()
    total   = float(n * 9)
    return float(correct / total) if total > 0 else 0.0


def train_ranking_model(train_X, train_R, val_X, val_R):
    """BPR training with cosine LR schedule + early stopping.
    Caches the trained model to BPR_MODEL_CACHE; skips retraining on rerun."""
    print(f"\nTraining ranking model on {DEVICE} ...")
    print(f"  Train: {train_X.shape[0]:,} groups  |  "
          f"Val: {val_X.shape[0]:,} groups")

    if os.path.exists(BPR_MODEL_CACHE):
        print(f"  Loading cached BPR model from "
              f"{os.path.basename(BPR_MODEL_CACHE)} ...")
        ckpt = torch.load(BPR_MODEL_CACHE, map_location=DEVICE,
                          weights_only=False)
        model = RankingNet(N_FEATURES, HIDDEN_SIZES, DROPOUT).to(DEVICE)
        model.load_state_dict(ckpt["state_dict"])
        best_auc = float(ckpt.get("best_auc", 0.0))
        print(f"  Loaded.  Cached val AUC: {best_auc:.4f}")
```

```python
        return model, best_auc

    # Move the entire training tensor to GPU once. Shuffling and batch
    # slicing are done on-GPU per epoch -- no DataLoader, no per-batch
    # CPU->GPU transfer, no Python overhead.
    train_X_gpu = torch.tensor(train_X, dtype=torch.float32, device=DEVICE)
    train_R_gpu = torch.tensor(train_R, dtype=torch.float32, device=DEVICE)
    n_train     = train_X_gpu.shape[0]

    model     = RankingNet(N_FEATURES, HIDDEN_SIZES, DROPOUT).to(DEVICE)
    optimizer = torch.optim.Adam(model.parameters(), lr=LR,
                                 weight_decay=WEIGHT_DECAY)
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
        optimizer, MAX_EPOCHS)

    best_auc          = 0.0
    best_state        = None
    patience_counter  = 0

    for epoch in range(MAX_EPOCHS):
        model.train()
        perm = torch.randperm(n_train, device=DEVICE)
        total_loss = 0.0
        n_batches  = 0
        for start in range(0, n_train, BATCH_SIZE):
            idx     = perm[start:start + BATCH_SIZE]
            x_batch = train_X_gpu[idx]
            r_batch = train_R_gpu[idx]
            scores  = model(x_batch)
            loss    = bpr_group_loss(scores, r_batch, HARD_NEG_TEMP)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
            total_loss += loss.item()
            n_batches  += 1
        scheduler.step()
        avg_loss = total_loss / n_batches

        if (epoch + 1) % 5 == 0 or epoch == 0:
            model.eval()
            with torch.no_grad():
                sample_x = torch.tensor(train_X[:2048], dtype=torch.float32,
                                        device=DEVICE)
                sample_scores = model(sample_x).cpu().numpy()
                tc, tt = 0, 0
                for i in range(sample_scores.shape[0]):
                    for p in range(3):
                        for n in range(3, 6):
                            if   sample_scores[i, p] >  sample_scores[i, n]:
                                tc += 1
                            elif sample_scores[i, p] == sample_scores[i, n]:
                                tc += 0.5
                            tt += 1
                train_acc = tc / tt

            val_auc = compute_val_auc(model, val_X, val_R)
            improved = ""
            if val_auc > best_auc:
                best_auc   = val_auc
                best_state = {k: v.cpu().clone()
                              for k, v in model.state_dict().items()}
                patience_counter = 0
                improved = " *BEST*"
            else:
                patience_counter += 5

            print(f"  Epoch {epoch+1:3d}/{MAX_EPOCHS}  "
                  f"loss={avg_loss:.4f}  "
                  f"train_acc={train_acc:.4f}  "
                  f"val_AUC={val_auc:.4f}{improved}")

            if patience_counter >= PATIENCE:
                print(f"  Early stopping at epoch {epoch+1}")
                break

    if best_state is not None:
        model.load_state_dict(best_state)
        model.to(DEVICE)
    print(f"  Best val AUC: {best_auc:.4f}")
```

```python
    torch.save({
        "state_dict":   model.state_dict(),
        "best_auc":     best_auc,
        "n_features":   N_FEATURES,
        "hidden_sizes": HIDDEN_SIZES,
        "dropout":      DROPOUT,
    }, BPR_MODEL_CACHE)
    print(f"  Cached -> {os.path.basename(BPR_MODEL_CACHE)}")
    return model, best_auc


# ====================================================================
# 10. Prediction & Submission
# ====================================================================

def predict_test(model, user_ratings, track_meta, test_users,
                 feat_mean, feat_std,
                 global_stats, album_to_tracks, artist_to_tracks,
                 mf_uid_map, mf_iid_map, mf_user_f, mf_item_f,
                 out_path, model_name="model"):
    """Score test candidates -> top-3 per user -> Kaggle CSV.

    Returns:
      results       : [(key, label)]   binary 0/1 for the CSV
      raw_scores    : {(uid, tid): float}   continuous NN score for stacking
    """
    print(f"\nPredicting with {model_name} ...")
    t0 = time.time()
    model.eval()
    results    = []
    raw_scores = {}

    with torch.no_grad():
        for uid, candidates in test_users:
            ratings = user_ratings.get(uid, {})
            u_mean, u_std, u_n = compute_user_profile(ratings)

            base_feats = []
            for tid in candidates:
                bf = compute_base_features(
                    ratings, track_meta, tid, u_mean, u_std, u_n,
                    global_stats, album_to_tracks, artist_to_tracks,
                    mf_uid_map, mf_iid_map, mf_user_f, mf_item_f, uid)
                base_feats.append(bf)

            cross = compute_cross_features(track_meta, candidates)

            X = np.zeros((1, 6, N_FEATURES), dtype=np.float32)
            for c in range(6):
                X[0, c, :N_BASE_FEATURES] = base_feats[c]
                X[0, c, N_BASE_FEATURES:] = cross[c]
            X = normalize(X, feat_mean, feat_std)

            X_t    = torch.tensor(X, dtype=torch.float32, device=DEVICE)
            scores = model(X_t).cpu().numpy()[0]  # shape (6,)

            for c, tid in enumerate(candidates):
                raw_scores[(uid, tid)] = float(scores[c])

            ranked = sorted(zip(scores, candidates), reverse=True)
            top3   = set(tid for _, tid in ranked[:3])
            for tid in candidates:
                results.append((f"{uid}_{tid}", 1 if tid in top3 else 0))

    with open(out_path, "w") as f:
        f.write("TrackID,Predictor\n")
        for key, label in results:
            f.write(f"{key},{label}\n")

    n_ones = sum(v for _, v in results)
    print(f"  {len(results):,} predictions  "
          f"({n_ones:,} rec / {len(results)-n_ones:,} not rec)  "
          f"[{elapsed(t0)}]")
    print(f"  Saved -> {os.path.basename(out_path)}")
    return results, raw_scores


# ====================================================================
# 11. V2 Heuristic Baseline
# ====================================================================
```

```python

def compute_v2_scores(uid, candidates, user_ratings, track_meta):
    """Per-candidate v2 score = album + artist + n_genre*mean_genre/10."""
    ratings = user_ratings.get(uid, {})
    scores  = []
    for tid in candidates:
        alb_id, art_id, genre_ids = track_meta.get(tid, (None, None, []))
        s = 0.0
        if alb_id is not None and alb_id in ratings:
            s += ratings[alb_id]
        if art_id is not None and art_id in ratings:
            s += ratings[art_id]
        gs = [ratings[g] for g in genre_ids if g in ratings]
        if gs:
            s += len(gs) * (sum(gs) / len(gs)) / 10.0
        scores.append(s)
    return scores


def predict_v2_baseline(user_ratings, track_meta, test_users, out_path):
    """Pure v2 heuristic submission (no neural network)."""
    print(f"\nGenerating v2 heuristic baseline ...")
    t0 = time.time()
    results = []
    for uid, candidates in test_users:
        v2_scores = compute_v2_scores(uid, candidates, user_ratings,
                                      track_meta)
        ranked = sorted(zip(v2_scores, candidates), reverse=True)
        top3   = set(tid for _, tid in ranked[:3])
        for tid in candidates:
            results.append((f"{uid}_{tid}", 1 if tid in top3 else 0))

    with open(out_path, "w") as f:
        f.write("TrackID,Predictor\n")
        for key, label in results:
            f.write(f"{key},{label}\n")
    n_ones = sum(v for _, v in results)
    print(f"  {len(results):,} predictions  "
          f"({n_ones:,} rec)  [{elapsed(t0)}]")
    print(f"  Saved -> {os.path.basename(out_path)}")
    return results


def v2_heuristic_val_auc(user_ratings, track_meta, val_groups):
    """Pair-AUC for v2 heuristic on the same val groups as the NN."""
    correct, total = 0, 0
    for uid, tids, _ in val_groups:
        scores = compute_v2_scores(uid, tids, user_ratings, track_meta)
        for p in range(3):
            for n in range(3, 6):
                if   scores[p] >  scores[n]: correct += 1
                elif scores[p] == scores[n]: correct += 0.5
                total += 1
    return correct / total if total > 0 else 0.0


# ====================================================================
# 11b. PySpark ALS (Final spec's expected method, used as one
#      meta-learner input column).
#
# Per Wang's supplemental "PySpark Recommendation Code for the Final
# Project.py", the ALS configuration is rank=5/maxIter=5 by default; HW8
# established that rank=20, maxIter=20 generalizes better on this dataset.
# We train ALS on the full ratings matrix and score every (uid, tid) pair
# we'll need: the 6,000 labeled rows AND the 120,000 Kaggle test rows.
# ====================================================================

def train_als_and_score(user_ratings, test_users, label_keys,
                        out_path_submission):
    """Train PySpark ALS on the full rating matrix, then score:
       (a) every (uid, tid) appearing in test2_new.txt   -- for stacking
       (b) every (uid, tid) in the Kaggle test set       -- for stacking
                                                            + ALS-only sub
    Returns:
       als_scores : {(uid, tid): float}   for ALL keys we care about
       None on failure (e.g., PySpark not installed)
    """
    section(f"PYSPARK ALS (rank={ALS_RANK})")

    # Cached scores -- skips Spark entirely on re-runs.
```

```python
    if os.path.exists(ALS_CACHE):
        print(f"  Loading cached ALS scores from "
              f"{os.path.basename(ALS_CACHE)} ...")
        z = np.load(ALS_CACHE, allow_pickle=True)
        keys   = z["keys"]
        values = z["values"]
        als_scores = {(int(k[0]), int(k[1])): float(v)
                      for k, v in zip(keys, values)}
        print(f"  Loaded {len(als_scores):,} ALS scores")
        # Also write the ALS-only submission from cached scores
        n_ones, n_total = 0, 0
        with open(out_path_submission, "w") as f:
            f.write("TrackID,Predictor\n")
            for uid, candidates in test_users:
                pairs = [(tid, als_scores.get((int(uid), int(tid)), 0.0))
                         for tid in candidates]
                top3 = set(t for t, _ in sorted(pairs, key=lambda x: x[1],
                                                reverse=True)[:3])
                for tid, _ in pairs:
                    pred = 1 if tid in top3 else 0
                    f.write(f"{uid}_{tid},{pred}\n")
                    n_ones  += pred
                    n_total += 1
        print(f"  ALS-only submission: {n_ones:,} rec / "
              f"{n_total-n_ones:,} not rec  -> "
              f"{os.path.basename(out_path_submission)}")
        return als_scores

    try:
        from pyspark.sql import SparkSession
        from pyspark.sql.types import IntegerType, FloatType, StructType, \
            StructField
        from pyspark.ml.recommendation import ALS
    except ImportError:
        print("  PySpark not installed -- skipping ALS column.")
        print("  pip install pyspark  to enable.")
        return None

    t0 = time.time()

    # Pin Spark's Python workers to the SAME interpreter that's running this
    # script. Otherwise Spark may spawn Python 3.14 workers from PATH, and
    # 3.14's socket finalization breaks the Spark<->Python protocol on
    # Windows (WinError 10038 / "An operation was attempted on something
    # that is not a socket").
    os.environ["PYSPARK_PYTHON"]        = sys.executable
    os.environ["PYSPARK_DRIVER_PYTHON"] = sys.executable
    print(f"  Pinning Spark workers to: {sys.executable}")

    # JVM thread stack bumped to 32M -- same fix as HW9 to avoid
    # StackOverflowError when serializing ALS factor matrices on Windows.
    spark = (SparkSession.builder
             .appName("EE627-Final-ALS")
             .config("spark.driver.memory",            "6g")
             .config("spark.driver.extraJavaOptions",  "-Xss32m")
             .config("spark.executor.extraJavaOptions","-Xss32m")
             .config("spark.pyspark.python",           sys.executable)
             .config("spark.pyspark.driver.python",    sys.executable)
             .getOrCreate())
    spark.sparkContext.setLogLevel("WARN")

    # Build training DataFrame from user_ratings dict
    print("  Assembling rating tuples ...")
    rows = []
    for uid, ratings in user_ratings.items():
        for iid, r in ratings.items():
            rows.append((int(uid), int(iid), float(r)))
    schema = StructType([
        StructField("userID", IntegerType()),
        StructField("itemID", IntegerType()),
        StructField("rating", FloatType()),
    ])
    train_df = spark.createDataFrame(rows, schema=schema)
    n_train  = train_df.count()
    print(f"  Training rows: {n_train:,}")

    print(f"  Training ALS (rank={ALS_RANK}, maxIter={ALS_ITERS}, "
          f"regParam={ALS_REG}) ...")
    als = ALS(
        rank=ALS_RANK,
```

```
        maxIter=ALS_ITERS,
        regParam=ALS_REG,
        userCol="userID",
        itemCol="itemID",
        ratingCol="rating",
        nonnegative=True,
        implicitPrefs=False,
        coldStartStrategy="drop",
        seed=42,
    )
    model = als.fit(train_df)
    print(f"  ALS trained  [{elapsed(t0)}]")

    # Build the union of all (uid, tid) we need to score:
    #   - test2_new.txt label rows  (for meta-learner training)
    #   - all 120K Kaggle test rows (for meta-learner inference + ALS sub)
    print("  Building score request set ...")
    all_keys = set(label_keys)
    for uid, candidates in test_users:
        for tid in candidates:
            all_keys.add((int(uid), int(tid)))

    score_rows = [(uid, tid) for uid, tid in all_keys]
    score_schema = StructType([
        StructField("userID", IntegerType()),
        StructField("itemID", IntegerType()),
    ])
    score_df = spark.createDataFrame(score_rows, schema=score_schema)

    print(f"  Scoring {len(score_rows):,} (uid, tid) pairs ...")
    pred_df = model.transform(score_df)
    pred_rows = pred_df.select("userID", "itemID", "prediction").collect()

    als_scores = {}
    n_drops = 0
    for r in pred_rows:
        uid = int(r["userID"])
        tid = int(r["itemID"])
        p   = r["prediction"]
        if p is None:        # cold-start -> coldStartStrategy='drop' filtered
            n_drops += 1
            continue
        als_scores[(uid, tid)] = float(p)

    # Backfill cold-start rows with the global mean rating
    if n_drops or len(als_scores) < len(score_rows):
        global_mean_row = train_df.selectExpr("avg(rating) as m").collect()[0]
        global_mean = float(global_mean_row["m"])
        n_filled = 0
        for k in score_rows:
            if k not in als_scores:
                als_scores[k] = global_mean
                n_filled += 1
        print(f"  Cold-start: {n_filled:,} pairs backfilled with global "
              f"mean ({global_mean:.2f})")

    # Write an ALS-only submission for diagnostic value
    if out_path_submission is not None:
        n_ones, n_total = 0, 0
        with open(out_path_submission, "w") as f:
            f.write("TrackID,Predictor\n")
            for uid, candidates in test_users:
                pairs = [(tid, als_scores[(uid, tid)]) for tid in candidates]
                top3  = set(t for t, _ in sorted(pairs, key=lambda x: x[1],
                                                 reverse=True)[:3])
                for tid, _ in pairs:
                    pred = 1 if tid in top3 else 0
                    f.write(f"{uid}_{tid},{pred}\n")
                    n_ones  += pred
                    n_total += 1
        print(f"  ALS-only submission: {n_ones:,} rec / "
              f"{n_total-n_ones:,} not rec  -> "
              f"{os.path.basename(out_path_submission)}")

    spark.stop()

    # Cache scores so re-runs skip Spark entirely
    keys_arr   = np.array(list(als_scores.keys()), dtype=np.int64)
    values_arr = np.array([als_scores[k] for k in als_scores.keys()],
                          dtype=np.float64)
```

```python
    np.savez(ALS_CACHE, keys=keys_arr, values=values_arr)
    print(f"  Cached ALS scores -> {os.path.basename(ALS_CACHE)}")

    print(f"  ALS pipeline complete  [{elapsed(t0)}]")
    return als_scores


# ====================================================================
# 12. Confidence-Gated Hybrid (the actual 0.911 producer)
# ====================================================================

def generate_hybrid_submissions(test_users, user_ratings, track_meta,
                                v5_results, thresholds, output_dir):
    """For each gap threshold, build a hybrid submission:
        if v2_gap > threshold:  use v2 top-3
        else:                   defer to neural net (v5)
    Writes one CSV per threshold."""
    section("CONFIDENCE-GATED HYBRID SWEEP")

    # Build v5 lookup by (uid, tid)
    v5_by_key = {key: label for key, label in v5_results}

    # Pre-compute v2 top-3 + gap per user
    user_data = []
    for uid, candidates in test_users:
        v2_scores = compute_v2_scores(uid, candidates, user_ratings,
                                      track_meta)
        ranked    = sorted(zip(v2_scores, candidates), reverse=True)
        v2_top3   = set(tid for _, tid in ranked[:3])
        ss        = sorted(v2_scores, reverse=True)
        gap       = ss[2] - ss[3]
        user_data.append((uid, candidates, v2_top3, gap))

    print(f"  {'Thresh':>6}  {'v2 users':>10}  {'v5 users':>10}  "
          f"{'v5 pct':>8}  File")
    print("  " + "-" * 70)

    for thresh in thresholds:
        rows = []
        n_v2, n_v5 = 0, 0
        for uid, candidates, v2_top3, gap in user_data:
            if gap > thresh:
                n_v2 += 1
                for tid in candidates:
                    key = f"{uid}_{tid}"
                    rows.append((key, 1 if tid in v2_top3 else 0))
            else:
                n_v5 += 1
                for tid in candidates:
                    key = f"{uid}_{tid}"
                    rows.append((key, v5_by_key.get(key, 0)))

        fname = f"submission_final_v5_hybrid_gap{thresh}.csv"
        fpath = os.path.join(output_dir, fname)
        with open(fpath, "w") as f:
            f.write("TrackID,Predictor\n")
            for key, label in rows:
                f.write(f"{key},{label}\n")

        v5_pct = 100 * n_v5 / len(user_data)
        marker = "  <- production (0.911 on midterm)" if thresh == 10 else ""
        print(f"  {thresh:>6}  {n_v2:>10,}  {n_v5:>10,}  "
              f"{v5_pct:>7.1f}%  {fname}{marker}")


# ====================================================================
# 12b. Stacked Meta-Learner
# ====================================================================
#
# Trains a logistic regression on test2_new.txt labels using whatever
# score signals are available:
#   - v5 NN raw score        (this script's output)
#   - v2 heuristic score     (this script's output)
#   - HW9 PySpark probabilities  (loaded from ../Homework 9/hw9_probabilities.csv
#                                 if present)
#
# Falls back to (v5, v2) only if HW9 probs are missing.  Falls back to
# hand-coded LR if sklearn is missing.

def build_v2_score_dict(test_users, user_ratings, track_meta):
```

```python
    """{(uid, tid): v2_score} for every test pair."""
    print("\nBuilding v2 score lookup for stacking ...")
    t0 = time.time()
    out = {}
    for uid, candidates in test_users:
        scores = compute_v2_scores(uid, candidates, user_ratings, track_meta)
        for tid, s in zip(candidates, scores):
            out[(uid, tid)] = float(s)
    print(f"  {len(out):,} (uid, tid) v2 scores  [{elapsed(t0)}]")
    return out


def load_hw9_probabilities(path):
    """Load HW9's per-(uid, tid) probability matrix if it exists.
    Returns dict {(uid, tid): {model_name: prob}} or None."""
    if not os.path.exists(path):
        return None
    print(f"\nLoading HW9 probability matrix ...")
    t0 = time.time()
    import csv as _csv
    out = {}
    model_names = None
    with open(path, "r", newline="") as f:
        reader = _csv.reader(f)
        header = next(reader)
        model_names = header[2:]   # skip user_id, track_id columns
        for row in reader:
            uid = int(row[0])
            tid = int(row[1])
            out[(uid, tid)] = {n: float(v)
                               for n, v in zip(model_names, row[2:])}
    print(f"  {len(out):,} rows x {len(model_names)} models loaded "
          f"[{elapsed(t0)}]")
    print(f"  Models: {', '.join(model_names)}")
    return out, model_names


def load_test2_labels(path):
    """test2_new.txt -> {(uid, tid): 0_or_1}."""
    labels = {}
    with open(path, "r") as f:
        for line in f:
            parts = line.strip().split("|")
            if len(parts) == 3:
                labels[(int(parts[0]), int(parts[1]))] = int(parts[2])
    return labels


def load_past_predictions(directory):
    """Load every submission CSV in `directory` as a binary 0/1 column.

    Filename convention: <name>_<score>.csv  (e.g., v5b_hybrid_gap10_0911.csv)
    The trailing 4 digits encode the Kaggle score (0911 -> 0.911).  This is
    used purely for logging / sorting -- the meta-learner sees only the
    binary predictions.

    Returns:
      preds : {(uid, tid): {col_name: 0_or_1}}
      cols  : ordered list of column names (sorted by Kaggle score desc)
    """
    if not os.path.isdir(directory):
        return None, []

    import csv as _csv
    files = [f for f in os.listdir(directory) if f.endswith(".csv")]
    if not files:
        return None, []

    print(f"\nLoading past predictions from {os.path.basename(directory)} ...")
    t0 = time.time()

    # Parse score from filename if present
    def parse_score(fname):
        stem = fname[:-4]   # drop .csv
        if "_" in stem:
            tail = stem.rsplit("_", 1)[1]
            if tail.isdigit() and len(tail) == 4:
                return int(tail) / 1000.0
        return None

```

```python
    files_with_scores = [(f, parse_score(f)) for f in files]
    # Sort: known scores desc, unknown last
    files_with_scores.sort(
        key=lambda x: (-x[1] if x[1] is not None else 1.0, x[0]))

    cols  = []
    preds = defaultdict(dict)
    for fname, score in files_with_scores:
        col_name = fname[:-4]
        cols.append(col_name)
        path = os.path.join(directory, fname)
        with open(path, "r", newline="") as f:
            reader = _csv.reader(f)
            next(reader)   # header
            for row in reader:
                key  = row[0]
                pred = int(row[1])
                # key format: "uid_tid"
                uid_str, tid_str = key.split("_")
                preds[(int(uid_str), int(tid_str))][col_name] = pred

    print(f"  Loaded {len(cols)} submissions  |  "
          f"{len(preds):,} (uid, tid) pairs  [{elapsed(t0)}]")
    print(f"  Score range:")
    for fname, score in files_with_scores:
        score_str = f"{score:.3f}" if score is not None else "  ?  "
        print(f"    {score_str}   {fname}")
    return preds, cols


def fit_logistic_regression(X, y, n_iter=2000, lr=0.05, l2=0.01):
    """Hand-coded LR with L2.  Used if sklearn is unavailable.
    X: (n, d), y: (n,) in {0, 1}.  Features assumed roughly z-scored."""
    n, d = X.shape
    w = np.zeros(d)
    b = 0.0
    for _ in range(n_iter):
        z   = X @ w + b
        p   = 1.0 / (1.0 + np.exp(-np.clip(z, -30, 30)))
        err = p - y
        gw  = (X.T @ err) / n + l2 * w
        gb  = err.mean()
        w  -= lr * gw
        b  -= lr * gb
    return w, b


def _rank_normalize_per_user(score_dict, test_users):
    """Convert absolute scores to per-user rank in [0, 1].

    For each user's 6 candidates, the highest-scoring tid gets 1.0 and
    the lowest gets 0.0.  This is the signal Kaggle actually scores on
    (top-3 within user), so feeding it explicitly helps the meta-learner.
    """
    out = {}
    for uid, candidates in test_users:
        scored = [(tid, score_dict.get((uid, tid), 0.0)) for tid in candidates]
        scored.sort(key=lambda x: x[1])
        n = len(scored)
        for rank, (tid, _) in enumerate(scored):
            out[(uid, tid)] = rank / max(n - 1, 1)
    return out


def _consensus_from_past(past_preds, past_cols, prefix):
    """Collapse near-duplicate past submissions (same family) into a
    single 'consensus rate' feature: fraction of family members that
    voted 1 for this (uid, tid).

    Returns: {(uid, tid): float in [0, 1]}, or None if no matches.
    """
    family = [c for c in past_cols if c.startswith(prefix)]
    if not family:
        return None, []
    out = {}
    for key, votes in past_preds.items():
        vals = [votes.get(c, 0) for c in family]
        out[key] = sum(vals) / len(family)
    return out, family
```

```python

# ====================================================================
# v4 helpers: Item-item collaborative filtering
# ====================================================================
# External method (cited in SOURCES.md):
#   - Item-item collaborative filtering with cosine similarity:
#       Sarwar et al. (2001); Linden, Smith & York (2003)
# ====================================================================

def build_item_inverted_index(user_ratings):
    """item_id -> {user_id: rating} dict-of-dicts.

    Used to compute item-item cosine similarity sparsely: for two items
    i and j, the dot product over users is sum over u in (U_i n U_j) of
    r_ui * r_uj.
    """
    item_users = defaultdict(dict)
    for uid, ratings in user_ratings.items():
        for tid, r in ratings.items():
            item_users[tid][uid] = float(r)
    # Pre-compute L2 norms once
    item_norms = {tid: math.sqrt(sum(v * v for v in users.values()))
                  for tid, users in item_users.items()}
    return dict(item_users), item_norms


def cosine_item_pair(i, j, item_users, item_norms, min_co=5):
    """Cosine similarity between item i and item j over their co-raters.

    Returns 0.0 if either item is unrated, or if they have fewer than
    `min_co` users in common (avoids spurious high-similarity from
    items with only 1-2 shared raters).
    """
    if i == j:
        return 1.0
    ni = item_norms.get(i, 0.0)
    nj = item_norms.get(j, 0.0)
    if ni == 0.0 or nj == 0.0:
        return 0.0
    ui = item_users.get(i, {})
    uj = item_users.get(j, {})
    # Iterate the smaller set
    if len(ui) > len(uj):
        ui, uj = uj, ui
    co  = 0
    dot = 0.0
    for u, r in ui.items():
        if u in uj:
            dot += r * uj[u]
            co  += 1
    if co < min_co:
        return 0.0
    return dot / (ni * nj)


def score_iicf(test_users, user_ratings, item_users, item_norms,
               topk=IICF_TOPK, min_co=IICF_MIN_CO):
    """For each (uid, tid) candidate, compute item-item CF score:
        score = sum_{j in user's top-K rated items} sim(tid, j) * r_uj

    Returns: {(uid, tid): float}
    """
    out = {}
    n_users = len(test_users)
    t0 = time.time()
    for u_idx, (uid, candidates) in enumerate(test_users):
        # Get user's history, take top-K by rating
        ratings = user_ratings.get(uid, {})
        if not ratings:
            for tid in candidates:
                out[(uid, tid)] = 0.0
            continue
        history = sorted(ratings.items(), key=lambda x: x[1],
                         reverse=True)[:topk]

        for tid in candidates:
            score = 0.0
            for hid, hr in history:
                sim = cosine_item_pair(tid, hid, item_users, item_norms,
                                       min_co=min_co)
```

```
                score += sim * hr
            out[(uid, tid)] = score

        if (u_idx + 1) % 2000 == 0:
            elapsed_s = time.time() - t0
            rate = (u_idx + 1) / elapsed_s
            eta  = (n_users - u_idx - 1) / rate
            print(f"    iicf scored {u_idx+1:,}/{n_users:,} users  "
                  f"(rate {rate:.0f} u/s, eta {eta:.0f}s)")

    print(f"  iicf scored {len(out):,} pairs  [{elapsed(t0)}]")
    return out


def compute_or_load_iicf(test_users, user_ratings,
                         topk=None, min_co=None,
                         item_index_cache=None):
    """Compute item-item CF scores for all 120K test pairs, with caching.

    Args:
      topk, min_co       : hyperparameters for this variant.  Default
                           falls back to the module-level IICF_TOPK /
                           IICF_MIN_CO (variant 'a').
      item_index_cache   : optional pre-built (item_users, item_norms)
                           tuple, so multiple variants share the index
                           (it costs ~5s and ~1GB to build).
    """
    if topk is None:
        topk = IICF_TOPK
    if min_co is None:
        min_co = IICF_MIN_CO

    cache_path = os.path.join(CACHE_DIR, f"iicf_k{topk}_m{min_co}.npz")
    if os.path.exists(cache_path):
        print(f"  Loading cached item-item CF scores from "
              f"{os.path.basename(cache_path)} ...")
        z = np.load(cache_path, allow_pickle=True)
        keys   = z["keys"]
        values = z["values"]
        out = {(int(k[0]), int(k[1])): float(v)
               for k, v in zip(keys, values)}
        print(f"  Loaded {len(out):,} iicf scores")
        return out

    if item_index_cache is None:
        print(f"  Building item inverted index ...")
        t0 = time.time()
        item_users, item_norms = build_item_inverted_index(user_ratings)
        print(f"  {len(item_users):,} items indexed  [{elapsed(t0)}]")
    else:
        item_users, item_norms = item_index_cache

    print(f"  Scoring item-item CF (top-{topk} history, "
          f"min_co={min_co}) ...")
    out = score_iicf(test_users, user_ratings, item_users, item_norms,
                     topk=topk, min_co=min_co)

    keys_arr   = np.array(list(out.keys()), dtype=np.int64)
    values_arr = np.array([out[k] for k in out.keys()], dtype=np.float64)
    np.savez(cache_path, keys=keys_arr, values=values_arr)
    print(f"  Cached -> {os.path.basename(cache_path)}")
    return out


def write_iicf_only_submission(test_users, iicf_scores, out_path):
    """Pure item-item CF top-3 per user, sanity-check submission."""
    n_ones, n_total = 0, 0
    with open(out_path, "w") as f:
        f.write("TrackID,Predictor\n")
        for uid, candidates in test_users:
            pairs = [(tid, iicf_scores.get((uid, tid), 0.0))
                     for tid in candidates]
            top3 = set(t for t, _ in sorted(pairs, key=lambda x: x[1],
                                            reverse=True)[:3])
            for tid, _ in pairs:
                pred = 1 if tid in top3 else 0
                f.write(f"{uid}_{tid},{pred}\n")
                n_ones  += pred
                n_total += 1
    print(f"  iicf-only submission: {n_ones:,} rec / "
```

```
          f"{n_total-n_ones:,} not rec  -> {os.path.basename(out_path)}")


# ====================================================================
# v3 helpers: LightGBM LambdaRank, isotonic calibration, NDCG@3 CV
# ====================================================================
# External methods (cited in SOURCES.md):
#   - LambdaMART / LightGBM LambdaRank: Burges (2010); Ke et al. (2017)
#   - Isotonic Regression calibration: Zadrozny & Elkan (2002);
#       Niculescu-Mizil & Caruana (2005)
#   - NDCG@K with grouped CV: Jarvelin & Kekalainen (2002)
# ====================================================================

def train_lgbm_lambdarank(train_X, val_X, cache_path):
    """Train a LightGBM LambdaRank model on the same group-of-6 features
    used by the BPR neural net.  Each group is a query with 6 documents;
    relevance grades are assigned by within-group rating rank
    [5, 4, 3, 2, 1, 0] (the groups are already sorted by rating desc -- so
    the actual rating values aren't needed here, only the position).

    Returns the fitted Booster.  Cached as a text dump for re-runs.
    """
    import lightgbm as lgb

    if os.path.exists(cache_path):
        print(f"  Loading cached LightGBM ranker from "
              f"{os.path.basename(cache_path)} ...")
        return lgb.Booster(model_file=cache_path)

    n_train, g_size, n_feat = train_X.shape
    n_val   = val_X.shape[0]

    # Flatten (N, 6, F) -> (N*6, F).  Groups are already sorted by rating,
    # so within-group rank = position.  Relevance grade = (5 - position).
    X_tr = train_X.reshape(n_train * g_size, n_feat)
    X_va = val_X.reshape(n_val * g_size, n_feat)
    grades = np.array([5, 4, 3, 2, 1, 0], dtype=np.int32)
    y_tr = np.tile(grades, n_train)
    y_va = np.tile(grades, n_val)
    grp_tr = np.full(n_train, g_size, dtype=np.int32)
    grp_va = np.full(n_val,   g_size, dtype=np.int32)

    print(f"  Training LightGBM LambdaRank "
          f"({n_train:,} groups x {g_size}) ...")
    t0 = time.time()
    train_ds = lgb.Dataset(X_tr, label=y_tr, group=grp_tr)
    val_ds   = lgb.Dataset(X_va, label=y_va, group=grp_va,
                           reference=train_ds)

    params = {
        "objective":      "lambdarank",
        "metric":         "ndcg",
        "ndcg_eval_at":   [3],
        "learning_rate":  0.05,
        "num_leaves":     63,
        "min_data_in_leaf": 50,
        "feature_fraction": 0.9,
        "bagging_fraction": 0.9,
        "bagging_freq":   5,
        "lambda_l2":      0.1,
        "verbose":        -1,
        "seed":           SEED,
        "num_threads":    -1,
    }

    booster = lgb.train(
        params, train_ds,
        num_boost_round=500,
        valid_sets=[val_ds],
        valid_names=["val"],
        callbacks=[lgb.early_stopping(stopping_rounds=20),
                   lgb.log_evaluation(period=50)],
    )

    booster.save_model(cache_path)
    print(f"  Saved LightGBM ranker -> {os.path.basename(cache_path)}  "
          f"[{elapsed(t0)}]")
    return booster


```

```python
def score_test_with_lgbm(booster, user_ratings, track_meta, candidate_pairs,
                         feat_mean, feat_std,
                         global_stats, album_to_tracks, artist_to_tracks,
                         mf_uid_map, mf_iid_map, mf_user_f, mf_item_f):
    """Score arbitrary (uid, tid) pairs with the LightGBM ranker.

    Featurizes per-user groups (using each user's full candidate set so
    cross-features match training), then queries the booster.

    Args:
      candidate_pairs : list of (uid, [tids])  groups of any size

    Returns:
      {(uid, tid): float}
    """
    out = {}
    n_feat = N_FEATURES

    for uid, candidates in candidate_pairs:
        ratings = user_ratings.get(uid, {})
        u_mean, u_std, u_n = compute_user_profile(ratings)

        base_feats = [
            compute_base_features(
                ratings, track_meta, tid, u_mean, u_std, u_n,
                global_stats, album_to_tracks, artist_to_tracks,
                mf_uid_map, mf_iid_map, mf_user_f, mf_item_f, uid)
            for tid in candidates
        ]
        cross = compute_cross_features(track_meta, candidates)

        n = len(candidates)
        X = np.zeros((n, n_feat), dtype=np.float32)
        for c in range(n):
            X[c, :N_BASE_FEATURES] = base_feats[c]
            X[c, N_BASE_FEATURES:] = cross[c]
        X = (X - feat_mean) / feat_std

        scores = booster.predict(X)
        for c, tid in enumerate(candidates):
            out[(uid, tid)] = float(scores[c])

    return out


def isotonic_calibrate_oof(scores_array, labels_array, groups_array,
                           n_splits=5):
    """Cross-fitted isotonic regression on a 1-D score array.

    Fits k folds of IsotonicRegression using GroupKFold, producing
    out-of-fold calibrated scores.  Also returns a final regressor
    fit on all data, for applying to unlabeled rows.

    Args:
      scores_array : (N,) raw scores for the labeled subset
      labels_array : (N,) 0/1 labels
      groups_array : (N,) user IDs (for GroupKFold)

    Returns:
      oof_calibrated : (N,) cross-fitted calibrated scores
      final_iso      : IsotonicRegression fit on all (scores, labels)
    """
    from sklearn.isotonic import IsotonicRegression
    from sklearn.model_selection import GroupKFold

    oof = np.zeros_like(scores_array, dtype=np.float64)
    gkf = GroupKFold(n_splits=n_splits)
    for tr, va in gkf.split(scores_array, labels_array, groups=groups_array):
        iso = IsotonicRegression(out_of_bounds="clip", y_min=0.0, y_max=1.0)
        iso.fit(scores_array[tr], labels_array[tr])
        oof[va] = iso.predict(scores_array[va])

    final_iso = IsotonicRegression(out_of_bounds="clip",
                                   y_min=0.0, y_max=1.0)
    final_iso.fit(scores_array, labels_array)
    return oof, final_iso


def ndcg_at_k_grouped(y_true, y_score, groups, k=3):
    """Mean per-user NDCG@k for binary labels.
```

```python

    Each user's NDCG@k is computed on their own slice of (y_true, y_score).
    Users with no positives are skipped (DCG ideal = 0 -> NaN).
    """
    from sklearn.metrics import ndcg_score
    # Group rows by user
    by_user = defaultdict(list)
    for i, g in enumerate(groups):
        by_user[g].append(i)
    aucs = []
    for _, idxs in by_user.items():
        yt = np.asarray(y_true)[idxs]
        ys = np.asarray(y_score)[idxs]
        if yt.sum() == 0:
            continue
        # ndcg_score expects 2-D arrays (queries, docs)
        aucs.append(ndcg_score(yt.reshape(1, -1), ys.reshape(1, -1), k=k))
    return float(np.mean(aucs))


def _write_topk_submission(by_user, out_path, k=3):
    """Top-k per user -> Kaggle CSV.  Returns (n_ones, n_total)."""
    n_ones, n_total = 0, 0
    with open(out_path, "w") as f:
        f.write("TrackID,Predictor\n")
        for uid, items in by_user.items():
            topk = set(t for t, _ in sorted(items, key=lambda x: x[1],
                                            reverse=True)[:k])
            for tid, _ in items:
                pred = 1 if tid in topk else 0
                f.write(f"{uid}_{tid},{pred}\n")
                n_ones  += pred
                n_total += 1
    return n_ones, n_total


def generate_stacked_submission_v2(test_users, v5_raw, v2_raw,
                                   label_path, hw9_probs_path,
                                   past_preds_dir, out_path,
                                   als_scores=None):
    """v2 of the stacked meta-learner.  Targets the issues that capped v1
    at 0.918:

      1. Per-user RANK features for v5/v2/ALS (not just absolute scores).
      2. Collapse past-submission families (v5b_hybrid_*, v6_hybrid_*,
         hw9_*) into 'consensus rate' columns instead of 39 binary cols.
      3. GroupKFold-by-user CV (current code leaks because the same user
         appears in train and val of a random 5-fold split).
      4. Non-negative least squares baseline as a sanity check vs the
         logistic regression.
    """
    section("STACKED META-LEARNER v2")

    if not os.path.exists(label_path):
        print(f"  test2_new.txt not found at {label_path} -- skipping")
        return None

    labels = load_test2_labels(label_path)
    print(f"  Loaded {len(labels):,} labels  "
          f"({sum(labels.values()):,} positives)")

    hw9 = load_hw9_probabilities(hw9_probs_path)
    if hw9 is not None:
        hw9_probs, hw9_models = hw9
    else:
        hw9_probs, hw9_models = None, []

    past_preds, past_cols = load_past_predictions(past_preds_dir)
    if past_preds is None:
        past_preds, past_cols = {}, []

    # ---- Build rank-normalized per-user features ----
    print("\n  Computing per-user rank features ...")
    v5_rank = _rank_normalize_per_user(v5_raw, test_users)
    v2_rank = _rank_normalize_per_user(v2_raw, test_users)
    als_rank = (_rank_normalize_per_user(als_scores, test_users)
                if als_scores is not None else None)

    # ---- Collapse past-submission families into consensus features ----
    print("  Collapsing past-submission families into consensus ...")
```

```python
    families = {
        "v5b_hybrid": "v5b_hybrid_consensus",
        "v6_hybrid":  "v6_hybrid_consensus",
        "hw9":        "hw9_consensus",
        "v3":         "v3_consensus",
        "p1":         "p1_consensus",
        "hw8":        "hw8_consensus",
    }
    consensus_feats = {}
    consensus_cols  = []
    used_past_cols  = set()
    for prefix, col_name in families.items():
        cdict, members = _consensus_from_past(past_preds, past_cols, prefix)
        if cdict is None:
            continue
        consensus_feats[col_name] = cdict
        consensus_cols.append(col_name)
        used_past_cols.update(members)
        print(f"    {col_name:<24} {len(members):>2} members")

    # Past submissions that don't fit any family get kept individually
    # (handful of one-offs).
    leftover_cols = [c for c in past_cols if c not in used_past_cols]
    if leftover_cols:
        print(f"    {len(leftover_cols)} ungrouped past cols kept as-is")

    # ---- Assemble feature columns ----
    has_als = als_scores is not None
    feature_cols = (
        ["v5_rank", "v2_rank"]
        + (["als_rank"] if has_als else [])
        + ["v5_score", "v2_score"]
        + (["als_score"] if has_als else [])
        + list(hw9_models)
        + consensus_cols
        + leftover_cols
    )
    print(f"\n  v2 feature set ({len(feature_cols)} cols):")
    print(f"    rank features : {2 + (1 if has_als else 0)}")
    print(f"    raw scores    : {2 + (1 if has_als else 0)}")
    print(f"    HW9 probs     : {len(hw9_models)}")
    print(f"    family cons.  : {len(consensus_cols)}")
    print(f"    leftover past : {len(leftover_cols)}")

    def build_row(uid, tid):
        row = [v5_rank.get((uid, tid), 0.5), v2_rank.get((uid, tid), 0.5)]
        if als_rank is not None:
            row.append(als_rank.get((uid, tid), 0.5))
        row.extend([v5_raw.get((uid, tid), 0.0),
                    v2_raw.get((uid, tid), 0.0)])
        if als_scores is not None:
            row.append(als_scores.get((uid, tid), 0.0))
        if hw9_probs is not None:
            hw9_row = hw9_probs.get((uid, tid))
            if hw9_row is None:
                row.extend([0.5] * len(hw9_models))
            else:
                row.extend([hw9_row[m] for m in hw9_models])
        for c in consensus_cols:
            row.append(consensus_feats[c].get((uid, tid), 0.5))
        votes = past_preds.get((uid, tid), {})
        for c in leftover_cols:
            row.append(float(votes.get(c, 0)))
        return row

    # ---- Training matrix ----
    X_rows, y_rows, group_rows, missing = [], [], [], 0
    for (uid, tid), y in labels.items():
        if (uid, tid) not in v5_raw or (uid, tid) not in v2_raw:
            missing += 1
            continue
        if hw9_probs is not None and (uid, tid) not in hw9_probs:
            missing += 1
            continue
        X_rows.append(build_row(uid, tid))
        y_rows.append(y)
        group_rows.append(uid)

    if missing:
        print(f"\n  Skipped {missing:,} labeled rows (missing scores)")
```

```python

X = np.array(X_rows, dtype=np.float64)
y = np.array(y_rows, dtype=np.float64)
groups = np.array(group_rows)
print(f"  Training matrix: {X.shape}  "
      f"({int(y.sum()):,} pos / {int(len(y)-y.sum()):,} neg, "
      f"{len(np.unique(groups)):,} unique users)")

# ---- Standardize ----
feat_mean = X.mean(axis=0)
feat_std  = X.std(axis=0)
feat_std[feat_std < 1e-8] = 1.0
Xn = (X - feat_mean) / feat_std

# ---- 1) GroupKFold logistic regression -- honest CV ----
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GroupKFold
from sklearn.metrics import roc_auc_score

Cs = [0.001, 0.01, 0.1, 1.0, 10.0]
print(f"\n  GroupKFold(5) by user -- honest CV  ({len(Cs)} C values)")
gkf = GroupKFold(n_splits=5)
cv_results = []
for C in Cs:
    fold_aucs = []
    for tr, va in gkf.split(Xn, y, groups=groups):
        clf = LogisticRegression(C=C, max_iter=2000,
                                 random_state=SEED)
        clf.fit(Xn[tr], y[tr])
        p = clf.predict_proba(Xn[va])[:, 1]
        fold_aucs.append(roc_auc_score(y[va], p))
    mean_auc = float(np.mean(fold_aucs))
    std_auc  = float(np.std(fold_aucs))
    cv_results.append((C, mean_auc, std_auc))
    print(f"    C={C:>6.3f}   AUC = {mean_auc:.4f} +/- {std_auc:.4f}")

best_C, best_auc, _ = max(cv_results, key=lambda r: r[1])
print(f"\n  Best C = {best_C}  (grouped CV AUC = {best_auc:.4f})")

clf_lr = LogisticRegression(C=best_C, max_iter=2000, random_state=SEED)
clf_lr.fit(Xn, y)
weights_lr = clf_lr.coef_[0]
bias_lr    = float(clf_lr.intercept_[0])

# ---- 2) Non-negative least squares sanity baseline ----
# NNLS on a small core: forces all weights >= 0, kills sign-flip
# collinearity tricks the LR can pull.
from scipy.optimize import nnls
core_names = ["v5_rank", "v2_rank"]
if has_als:
    core_names.append("als_rank")
for m in hw9_models:
    core_names.append(m)
for c in consensus_cols:
    core_names.append(c)
core_idx = [feature_cols.index(c) for c in core_names if c in feature_cols]
Xn_core  = Xn[:, core_idx]

nnls_w, _ = nnls(np.column_stack([Xn_core, np.ones(len(y))]), y)
nnls_weights = nnls_w[:-1]
nnls_bias    = float(nnls_w[-1])
# Honest CV for NNLS too
fold_aucs = []
for tr, va in gkf.split(Xn_core, y, groups=groups):
    w_fold, _ = nnls(np.column_stack([Xn_core[tr],
                                      np.ones(len(tr))]), y[tr])
    p = Xn_core[va] @ w_fold[:-1] + w_fold[-1]
    fold_aucs.append(roc_auc_score(y[va], p))
nnls_cv_auc = float(np.mean(fold_aucs))
print(f"\n  NNLS baseline ({len(core_idx)} core feats)  "
      f"grouped CV AUC = {nnls_cv_auc:.4f}")

# ---- Print learned weights ----
print(f"\n  v2 LR weights (top 20 by abs):")
print(f"  {'Feature':<28} {'Weight':>10}")
print(f"  {'-'*28} {'-'*10}")
ranked = sorted(enumerate(weights_lr), key=lambda x: abs(x[1]),
                reverse=True)[:20]
for idx, w_val in ranked:
    print(f"  {feature_cols[idx]:<28} {w_val:>+10.4f}")
```

```python
    print(f"  {'(intercept)':<28} {bias_lr:>+10.4f}")

    print(f"\n  NNLS weights (non-negative core):")
    print(f"  {'Feature':<28} {'Weight':>10}")
    print(f"  {'-'*28} {'-'*10}")
    for name, idx in zip(core_names, range(len(core_idx))):
        print(f"  {name:<28} {nnls_weights[idx]:>+10.4f}")
    print(f"  {'(intercept)':<28} {nnls_bias:>+10.4f}")

    # ---- Score test rows with the LR (best CV winner) ----
    print(f"\n  Scoring all 120K test rows with v2 LR ...")
    by_user = defaultdict(list)
    for uid, candidates in test_users:
        for tid in candidates:
            row = np.array(build_row(uid, tid))
            row_n = (row - feat_mean) / feat_std
            score = float(row_n @ weights_lr + bias_lr)
            by_user[uid].append((tid, score))

    n_ones, n_total = 0, 0
    with open(out_path, "w") as f:
        f.write("TrackID,Predictor\n")
        for uid, items in by_user.items():
            top3 = set(t for t, _ in sorted(items, key=lambda x: x[1],
                                            reverse=True)[:3])
            for tid, _ in items:
                pred = 1 if tid in top3 else 0
                f.write(f"{uid}_{tid},{pred}\n")
                n_ones  += pred
                n_total += 1

    print(f"  {n_total:,} predictions  "
          f"({n_ones:,} rec / {n_total-n_ones:,} not rec)")
    print(f"  Saved -> {os.path.basename(out_path)}")

    # ---- Also save NNLS submission for comparison ----
    nnls_path = out_path.replace(".csv", "_nnls.csv")
    print(f"\n  Scoring all 120K test rows with NNLS core ...")
    by_user_n = defaultdict(list)
    for uid, candidates in test_users:
        for tid in candidates:
            row = np.array(build_row(uid, tid))
            row_n = (row - feat_mean) / feat_std
            row_core = row_n[core_idx]
            score = float(row_core @ nnls_weights + nnls_bias)
            by_user_n[uid].append((tid, score))
    n_ones, n_total = 0, 0
    with open(nnls_path, "w") as f:
        f.write("TrackID,Predictor\n")
        for uid, items in by_user_n.items():
            top3 = set(t for t, _ in sorted(items, key=lambda x: x[1],
                                            reverse=True)[:3])
            for tid, _ in items:
                pred = 1 if tid in top3 else 0
                f.write(f"{uid}_{tid},{pred}\n")
                n_ones  += pred
                n_total += 1
    print(f"  {n_total:,} predictions  "
          f"({n_ones:,} rec / {n_total-n_ones:,} not rec)")
    print(f"  Saved -> {os.path.basename(nnls_path)}")

    return out_path


def generate_stacked_submission_v3(test_users, v5_raw, v2_raw,
                                   train_X, val_X,
                                   user_ratings, track_meta,
                                   feat_mean, feat_std,
                                   global_stats, album_to_tracks,
                                   artist_to_tracks,
                                   mf_uid_map, mf_iid_map,
                                   mf_user_f, mf_item_f,
                                   label_path, hw9_probs_path,
                                   past_preds_dir, out_path,
                                   als_scores=None):
    """v3 stacker.  Three external methods (cited in SOURCES.md):

      A. LightGBM LambdaRank as a new base model  -> 'lgbm_rank_score'
      B. Isotonic Regression calibration of HW9 + v5 prob columns
      C. NDCG@3 with GroupKFold-by-user CV for tuning the meta-learner
```

```
    Writes 4 submissions (and an ablation table to the log):
      out_path                          -- v3 with all three methods
      out_path.replace -> _A_lgbm.csv     -- v2 features + LightGBM only
      out_path.replace -> _B_iso.csv      -- v2 features + isotonic only
      out_path.replace -> _C_ndcg.csv     -- v2 features tuned by NDCG@3
    """
    section("STACKED META-LEARNER v3 (LightGBM + Isotonic + NDCG@3)")

    if not os.path.exists(label_path):
        print(f"  test2_new.txt not found at {label_path} -- skipping")
        return None

    labels = load_test2_labels(label_path)
    print(f"  Loaded {len(labels):,} labels  "
          f"({sum(labels.values()):,} positives)")

    hw9 = load_hw9_probabilities(hw9_probs_path)
    if hw9 is not None:
        hw9_probs, hw9_models = hw9
    else:
        hw9_probs, hw9_models = None, []

    past_preds, past_cols = load_past_predictions(past_preds_dir)
    if past_preds is None:
        past_preds, past_cols = {}, []

    # ================================================================
    # Method A: LightGBM LambdaRank base model
    # ================================================================
    print("\n  [A] Training LightGBM LambdaRank base model ...")
    booster = train_lgbm_lambdarank(train_X, val_X, LGBM_CACHE)

    print("  [A] Scoring all 120K test rows + 6K labeled rows ...")
    t_lgbm = time.time()
    lgbm_scores = score_test_with_lgbm(
        booster, user_ratings, track_meta, test_users,
        feat_mean, feat_std,
        global_stats, album_to_tracks, artist_to_tracks,
        mf_uid_map, mf_iid_map, mf_user_f, mf_item_f)
    print(f"  [A] LightGBM scored {len(lgbm_scores):,} pairs  "
          f"[{elapsed(t_lgbm)}]")

    # ================================================================
    # v2-style feature scaffolding (rank features + family consensus)
    # ================================================================
    v5_rank = _rank_normalize_per_user(v5_raw, test_users)
    v2_rank = _rank_normalize_per_user(v2_raw, test_users)
    als_rank = (_rank_normalize_per_user(als_scores, test_users)
                if als_scores is not None else None)
    lgbm_rank = _rank_normalize_per_user(lgbm_scores, test_users)

    families = {
        "v5b_hybrid": "v5b_hybrid_consensus",
        "v6_hybrid":  "v6_hybrid_consensus",
        "hw9":        "hw9_consensus",
        "v3":         "v3_consensus",
        "p1":         "p1_consensus",
        "hw8":        "hw8_consensus",
    }
    consensus_feats = {}
    consensus_cols  = []
    used_past_cols  = set()
    for prefix, col_name in families.items():
        cdict, members = _consensus_from_past(past_preds, past_cols, prefix)
        if cdict is None:
            continue
        consensus_feats[col_name] = cdict
        consensus_cols.append(col_name)
        used_past_cols.update(members)
    leftover_cols = [c for c in past_cols if c not in used_past_cols]

    # ================================================================
    # Method B: Isotonic calibration of HW9 + v5 probability columns
    # ================================================================
    print("\n  [B] Building labeled subset for calibration fitting ...")
    label_keys = list(labels.keys())
    label_groups = np.array([uid for uid, _ in label_keys])
    label_y      = np.array([labels[k] for k in label_keys], dtype=np.float64)

```

```
    # Drop labeled rows missing any required score
    keep_mask = np.array([
        (k in v5_raw) and (k in v2_raw)
        and (hw9_probs is None or k in hw9_probs)
        for k in label_keys])
    keep_keys   = [k for k, m in zip(label_keys, keep_mask) if m]
    keep_groups = label_groups[keep_mask]
    keep_y      = label_y[keep_mask]
    print(f"  [B] Calibration set: {len(keep_keys):,} of "
          f"{len(label_keys):,} labels usable")

    # Calibrate v5 NN raw scores (continuous, well-suited to isotonic)
    v5_arr = np.array([v5_raw[k] for k in keep_keys], dtype=np.float64)
    _, iso_v5 = isotonic_calibrate_oof(v5_arr, keep_y, keep_groups)

    # Calibrate each HW9 probability column independently
    hw9_isos = {}
    if hw9_probs is not None:
        for m in hw9_models:
            arr = np.array([hw9_probs[k][m] for k in keep_keys],
                           dtype=np.float64)
            _, iso = isotonic_calibrate_oof(arr, keep_y, keep_groups)
            hw9_isos[m] = iso
        print(f"  [B] Fit {len(hw9_isos)} HW9 isotonic calibrators "
              f"+ 1 v5 calibrator")

    def calibrated_v5(uid, tid):
        return float(iso_v5.predict([v5_raw.get((uid, tid), 0.0)])[0])

    def calibrated_hw9(uid, tid, m):
        if hw9_probs is None:
            return 0.5
        row = hw9_probs.get((uid, tid))
        if row is None:
            return 0.5
        return float(hw9_isos[m].predict([row[m]])[0])

    # ================================================================
    # Build feature columns (per-variant) and score functions
    # ================================================================
    has_als = als_scores is not None

    # ---- v3 ALL: rank feats + LightGBM rank + isotonic HW9/v5 ----
    cols_all = (
        ["v5_rank", "v2_rank"]
        + (["als_rank"] if has_als else [])
        + ["lgbm_rank", "lgbm_score"]
        + ["v5_iso", "v2_score"]
        + (["als_score"] if has_als else [])
        + [f"{m}_iso" for m in hw9_models]
        + consensus_cols
        + leftover_cols
    )

    def row_all(uid, tid):
        row = [v5_rank.get((uid, tid), 0.5),
               v2_rank.get((uid, tid), 0.5)]
        if als_rank is not None:
            row.append(als_rank.get((uid, tid), 0.5))
        row += [lgbm_rank.get((uid, tid), 0.5),
                lgbm_scores.get((uid, tid), 0.0)]
        row += [calibrated_v5(uid, tid),
                v2_raw.get((uid, tid), 0.0)]
        if als_scores is not None:
            row.append(als_scores.get((uid, tid), 0.0))
        for m in hw9_models:
            row.append(calibrated_hw9(uid, tid, m))
        for c in consensus_cols:
            row.append(consensus_feats[c].get((uid, tid), 0.5))
        votes = past_preds.get((uid, tid), {})
        for c in leftover_cols:
            row.append(float(votes.get(c, 0)))
        return row

    # ---- variants A/B/C use selective feature sets ----
    cols_v2 = (   # baseline matching v2 (no LGBM, no isotonic)
        ["v5_rank", "v2_rank"]
        + (["als_rank"] if has_als else [])
        + ["v5_score", "v2_score"]
        + (["als_score"] if has_als else [])
```

```
        + list(hw9_models)
        + consensus_cols
        + leftover_cols
    )

    def row_v2(uid, tid):
        row = [v5_rank.get((uid, tid), 0.5),
               v2_rank.get((uid, tid), 0.5)]
        if als_rank is not None:
            row.append(als_rank.get((uid, tid), 0.5))
        row += [v5_raw.get((uid, tid), 0.0),
                v2_raw.get((uid, tid), 0.0)]
        if als_scores is not None:
            row.append(als_scores.get((uid, tid), 0.0))
        if hw9_probs is not None:
            hw9_row = hw9_probs.get((uid, tid))
            if hw9_row is None:
                row += [0.5] * len(hw9_models)
            else:
                row += [hw9_row[m] for m in hw9_models]
        for c in consensus_cols:
            row.append(consensus_feats[c].get((uid, tid), 0.5))
        votes = past_preds.get((uid, tid), {})
        for c in leftover_cols:
            row.append(float(votes.get(c, 0)))
        return row

    cols_A = cols_v2 + ["lgbm_rank", "lgbm_score"]
    def row_A(uid, tid):
        return row_v2(uid, tid) + [lgbm_rank.get((uid, tid), 0.5),
                                   lgbm_scores.get((uid, tid), 0.0)]

    cols_B = (    # v2 with HW9/v5 isotonic-calibrated
        ["v5_rank", "v2_rank"]
        + (["als_rank"] if has_als else [])
        + ["v5_iso", "v2_score"]
        + (["als_score"] if has_als else [])
        + [f"{m}_iso" for m in hw9_models]
        + consensus_cols
        + leftover_cols
    )

    def row_B(uid, tid):
        row = [v5_rank.get((uid, tid), 0.5),
               v2_rank.get((uid, tid), 0.5)]
        if als_rank is not None:
            row.append(als_rank.get((uid, tid), 0.5))
        row += [calibrated_v5(uid, tid),
                v2_raw.get((uid, tid), 0.0)]
        if als_scores is not None:
            row.append(als_scores.get((uid, tid), 0.0))
        for m in hw9_models:
            row.append(calibrated_hw9(uid, tid, m))
        for c in consensus_cols:
            row.append(consensus_feats[c].get((uid, tid), 0.5))
        votes = past_preds.get((uid, tid), {})
        for c in leftover_cols:
            row.append(float(votes.get(c, 0)))
        return row

    # cols_C = cols_v2 (same features as v2; only the CV metric changes)

    # ================================================================
    # Method C: NDCG@3-grouped CV.  Sweep C over each variant.
    # ================================================================
    from sklearn.linear_model import LogisticRegression
    from sklearn.model_selection import GroupKFold
    from sklearn.metrics import roc_auc_score

    Cs = [0.001, 0.01, 0.1, 1.0, 10.0]
    gkf = GroupKFold(n_splits=5)

    def build_matrix(row_fn):
        Xs, ys, gs, miss = [], [], [], 0
        for k, y in labels.items():
            uid, tid = k
            if (uid, tid) not in v5_raw or (uid, tid) not in v2_raw:
                miss += 1
                continue
            if hw9_probs is not None and (uid, tid) not in hw9_probs:
```

```
                miss += 1
                continue
            Xs.append(row_fn(uid, tid))
            ys.append(y)
            gs.append(uid)
        X = np.array(Xs, dtype=np.float64)
        y = np.array(ys, dtype=np.float64)
        g = np.array(gs)
        # standardize
        mu, sd = X.mean(axis=0), X.std(axis=0)
        sd[sd < 1e-8] = 1.0
        return X, (X - mu) / sd, y, g, mu, sd, miss

    def cv_sweep(Xn, y, g, metric):
        results = []
        for C in Cs:
            fold_scores = []
            for tr, va in gkf.split(Xn, y, groups=g):
                clf = LogisticRegression(C=C, max_iter=2000,
                                         random_state=SEED)
                clf.fit(Xn[tr], y[tr])
                p = clf.predict_proba(Xn[va])[:, 1]
                if metric == "auc":
                    fold_scores.append(roc_auc_score(y[va], p))
                else:
                    fold_scores.append(
                        ndcg_at_k_grouped(y[va], p, g[va], k=3))
            results.append((C, float(np.mean(fold_scores)),
                            float(np.std(fold_scores))))
        return results

    def fit_and_score_variant(variant_name, cols, row_fn,
                              metric, sub_path):
        print(f"\n  --- Variant {variant_name} ({len(cols)} feats, "
              f"CV metric = {metric.upper()}) ---")
        X, Xn, y, g, mu, sd, miss = build_matrix(row_fn)
        if miss:
            print(f"    Skipped {miss:,} labeled rows (missing scores)")
        print(f"    Training matrix: {X.shape}  "
              f"({int(y.sum()):,} pos / {int(len(y)-y.sum()):,} neg)")
        sweep = cv_sweep(Xn, y, g, metric)
        for C, m, s in sweep:
            print(f"    C={C:>6.3f}   {metric.upper()} = "
                  f"{m:.4f} +/- {s:.4f}")
        best_C, best_m, _ = max(sweep, key=lambda r: r[1])
        # Also report the *other* metric at the best C
        clf = LogisticRegression(C=best_C, max_iter=2000,
                                 random_state=SEED)
        clf.fit(Xn, y)

        # Score test rows
        by_user = defaultdict(list)
        for uid, candidates in test_users:
            for tid in candidates:
                row = (np.array(row_fn(uid, tid)) - mu) / sd
                p = float(clf.decision_function(row.reshape(1, -1))[0])
                by_user[uid].append((tid, p))
        n_ones, n_total = _write_topk_submission(by_user, sub_path)
        print(f"    Best C = {best_C}  "
              f"({metric.upper()} = {best_m:.4f})")
        print(f"    {n_total:,} preds  ({n_ones:,} rec)  "
              f"-> {os.path.basename(sub_path)}")
        return best_m, best_C

    # ---- Run all four variants ----
    print("\n  ============ ABLATION ============")
    print("  Each variant fits on 6K labels, GroupKFold(5) by user.")
    print("  CV scores are HONEST (no user appears in both folds).")

    base_path = out_path
    a_path = base_path.replace(".csv", "_A_lgbm.csv")
    b_path = base_path.replace(".csv", "_B_iso.csv")
    c_path = base_path.replace(".csv", "_C_ndcg.csv")

    # Variant V2 baseline (for reference, AUC metric)
    auc_v2, _ = fit_and_score_variant(
        "V2 reference (AUC-tuned)", cols_v2, row_v2, "auc",
        base_path.replace(".csv", "_V2ref.csv"))

    auc_A, _ = fit_and_score_variant(
```

```
        "A: V2 + LightGBM only", cols_A, row_A, "auc", a_path)

    auc_B, _ = fit_and_score_variant(
        "B: V2 + Isotonic only", cols_B, row_B, "auc", b_path)

    ndcg_C, _ = fit_and_score_variant(
        "C: V2 features + NDCG@3 tuning", cols_v2, row_v2, "ndcg", c_path)

    ndcg_all, _ = fit_and_score_variant(
        "ALL: LightGBM + Isotonic + NDCG@3", cols_all, row_all,
        "ndcg", base_path)

    # ---- Ablation table ----
    print("\n  ============ ABLATION SUMMARY ============")
    print(f"  {'Variant':<40} {'CV metric':>12}")
    print(f"  {'-'*40} {'-'*12}")
    print(f"  {'V2 reference (AUC-tuned)':<40} {auc_v2:>12.4f} AUC")
    print(f"  {'A: + LightGBM only (AUC-tuned)':<40} {auc_A:>12.4f} AUC")
    print(f"  {'B: + Isotonic only (AUC-tuned)':<40} {auc_B:>12.4f} AUC")
    print(f"  {'C: V2 features + NDCG@3-tuned':<40} "
          f"{ndcg_C:>12.4f} NDCG@3")
    print(f"  {'ALL: A+B+C combined':<40} "
          f"{ndcg_all:>12.4f} NDCG@3")
    print(f"\n  NOTE: AUC and NDCG@3 are different scales;")
    print(f"        compare (V2,A,B) against each other and (C,ALL)")
    print(f"        against each other.")

    return base_path


def generate_stacked_submission_v4(test_users, v5_raw, v2_raw, iicf_scores,
                                   label_path, hw9_probs_path,
                                   past_preds_dir, out_path,
                                   als_scores=None):
    """v4 stacker: v3 'ALL' features (per-user ranks, family consensus,
    isotonic-calibrated HW9/v5) + new item-item CF column.  NDCG@3 CV.

    Item-item CF (Sarwar 2001; Linden 2003) is genuinely orthogonal to
    the MF/BPR/RF stack -- it's a pure neighborhood method on the raw
    rating matrix, no learned embeddings.  This is the diversity signal
    we expect to push past the 0.919 plateau.

    Writes:
      out_path                        -- v4 main: ALL v3 features + iicf
      out_path -> _iicf_only.csv      -- pure iicf top-3 per user
                                         (sanity check, like ALS-only)
    """
    section("STACKED META-LEARNER v4 (item-item CF + v3 features)")

    if not os.path.exists(label_path):
        print(f"  test2_new.txt not found -- skipping")
        return None

    labels = load_test2_labels(label_path)
    print(f"  Loaded {len(labels):,} labels  "
          f"({sum(labels.values()):,} positives)")

    hw9 = load_hw9_probabilities(hw9_probs_path)
    if hw9 is not None:
        hw9_probs, hw9_models = hw9
    else:
        hw9_probs, hw9_models = None, []

    past_preds, past_cols = load_past_predictions(past_preds_dir)
    if past_preds is None:
        past_preds, past_cols = {}, []

    # Per-user ranks for all the score columns
    v5_rank   = _rank_normalize_per_user(v5_raw, test_users)
    v2_rank   = _rank_normalize_per_user(v2_raw, test_users)
    iicf_rank = _rank_normalize_per_user(iicf_scores, test_users)
    als_rank  = (_rank_normalize_per_user(als_scores, test_users)
                 if als_scores is not None else None)

    # Family consensus features
    families = {
        "v5b_hybrid": "v5b_hybrid_consensus",
        "v6_hybrid":  "v6_hybrid_consensus",
        "hw9":        "hw9_consensus",
        "v3":         "v3_consensus",
```

```
        "p1":         "p1_consensus",
        "hw8":        "hw8_consensus",
    }
    consensus_feats = {}
    consensus_cols  = []
    used_past_cols  = set()
    for prefix, col_name in families.items():
        cdict, members = _consensus_from_past(past_preds, past_cols, prefix)
        if cdict is None:
            continue
        consensus_feats[col_name] = cdict
        consensus_cols.append(col_name)
        used_past_cols.update(members)
    leftover_cols = [c for c in past_cols if c not in used_past_cols]

    # Isotonic calibration (reuse v3's helper)
    print("\n  Fitting isotonic calibrators on labeled subset ...")
    label_keys = list(labels.keys())
    label_groups = np.array([uid for uid, _ in label_keys])
    label_y      = np.array([labels[k] for k in label_keys],
                            dtype=np.float64)
    keep_mask = np.array([
        (k in v5_raw) and (k in v2_raw)
        and (hw9_probs is None or k in hw9_probs)
        for k in label_keys])
    keep_keys   = [k for k, m in zip(label_keys, keep_mask) if m]
    keep_groups = label_groups[keep_mask]
    keep_y      = label_y[keep_mask]

    v5_arr = np.array([v5_raw[k] for k in keep_keys], dtype=np.float64)
    _, iso_v5 = isotonic_calibrate_oof(v5_arr, keep_y, keep_groups)
    hw9_isos = {}
    if hw9_probs is not None:
        for m in hw9_models:
            arr = np.array([hw9_probs[k][m] for k in keep_keys],
                           dtype=np.float64)
            _, iso = isotonic_calibrate_oof(arr, keep_y, keep_groups)
            hw9_isos[m] = iso

    def calibrated_v5(uid, tid):
        return float(iso_v5.predict([v5_raw.get((uid, tid), 0.0)])[0])

    def calibrated_hw9(uid, tid, m):
        if hw9_probs is None:
            return 0.5
        row = hw9_probs.get((uid, tid))
        if row is None:
            return 0.5
        return float(hw9_isos[m].predict([row[m]])[0])

    has_als = als_scores is not None

    # ---- Feature columns ----
    feature_cols = (
        ["v5_rank", "v2_rank", "iicf_rank"]
        + (["als_rank"] if has_als else [])
        + ["v5_iso", "v2_score", "iicf_score"]
        + (["als_score"] if has_als else [])
        + [f"{m}_iso" for m in hw9_models]
        + consensus_cols
        + leftover_cols
    )
    print(f"\n  v4 feature set ({len(feature_cols)} cols):")
    print(f"    rank features : {3 + (1 if has_als else 0)} "
          f"(v5, v2, iicf{', als' if has_als else ''})")
    print(f"    raw scores    : {3 + (1 if has_als else 0)}")
    print(f"    HW9 isotonic  : {len(hw9_models)}")
    print(f"    family cons.  : {len(consensus_cols)}")
    print(f"    leftover past : {len(leftover_cols)}")

    def build_row(uid, tid):
        row = [v5_rank.get((uid, tid), 0.5),
               v2_rank.get((uid, tid), 0.5),
               iicf_rank.get((uid, tid), 0.5)]
        if als_rank is not None:
            row.append(als_rank.get((uid, tid), 0.5))
        row += [calibrated_v5(uid, tid),
                v2_raw.get((uid, tid), 0.0),
                iicf_scores.get((uid, tid), 0.0)]
        if als_scores is not None:
```

```
            row.append(als_scores.get((uid, tid), 0.0))
        for m in hw9_models:
            row.append(calibrated_hw9(uid, tid, m))
        for c in consensus_cols:
            row.append(consensus_feats[c].get((uid, tid), 0.5))
        votes = past_preds.get((uid, tid), {})
        for c in leftover_cols:
            row.append(float(votes.get(c, 0)))
        return row

    # ---- Training matrix ----
    X_rows, y_rows, group_rows, missing = [], [], [], 0
    for (uid, tid), y in labels.items():
        if (uid, tid) not in v5_raw or (uid, tid) not in v2_raw:
            missing += 1
            continue
        if hw9_probs is not None and (uid, tid) not in hw9_probs:
            missing += 1
            continue
        X_rows.append(build_row(uid, tid))
        y_rows.append(y)
        group_rows.append(uid)
    if missing:
        print(f"\n  Skipped {missing:,} labeled rows (missing scores)")

    X = np.array(X_rows, dtype=np.float64)
    y = np.array(y_rows, dtype=np.float64)
    groups = np.array(group_rows)
    print(f"  Training matrix: {X.shape}  "
          f"({int(y.sum()):,} pos / {int(len(y)-y.sum()):,} neg, "
          f"{len(np.unique(groups)):,} unique users)")

    feat_mean = X.mean(axis=0)
    feat_std  = X.std(axis=0)
    feat_std[feat_std < 1e-8] = 1.0
    Xn = (X - feat_mean) / feat_std

    # ---- GroupKFold sweep, NDCG@3 + AUC ----
    from sklearn.linear_model import LogisticRegression
    from sklearn.model_selection import GroupKFold
    from sklearn.metrics import roc_auc_score
    Cs = [0.001, 0.01, 0.1, 1.0, 10.0]
    gkf = GroupKFold(n_splits=5)

    print(f"\n  GroupKFold(5) by user -- NDCG@3 + AUC sweep")
    print(f"  {'C':>8}   {'NDCG@3':>8}   {'AUC':>8}")
    print(f"  {'-'*8}   {'-'*8}   {'-'*8}")
    sweep = []
    for C in Cs:
        ndcgs, aucs = [], []
        for tr, va in gkf.split(Xn, y, groups=groups):
            clf = LogisticRegression(C=C, max_iter=2000, random_state=SEED)
            clf.fit(Xn[tr], y[tr])
            p = clf.predict_proba(Xn[va])[:, 1]
            aucs.append(roc_auc_score(y[va], p))
            ndcgs.append(ndcg_at_k_grouped(y[va], p, groups[va], k=3))
        m_ndcg = float(np.mean(ndcgs))
        m_auc  = float(np.mean(aucs))
        sweep.append((C, m_ndcg, m_auc))
        print(f"  {C:>8.3f}   {m_ndcg:>8.4f}   {m_auc:>8.4f}")

    best_C, best_ndcg, best_auc = max(sweep, key=lambda r: r[1])
    print(f"\n  Best C = {best_C}  "
          f"(NDCG@3 = {best_ndcg:.4f}, AUC = {best_auc:.4f})")

    clf = LogisticRegression(C=best_C, max_iter=2000, random_state=SEED)
    clf.fit(Xn, y)
    weights = clf.coef_[0]
    bias    = float(clf.intercept_[0])

    print(f"\n  v4 LR weights (top 20 by abs):")
    print(f"  {'Feature':<28} {'Weight':>10}")
    print(f"  {'-'*28} {'-'*10}")
    ranked = sorted(enumerate(weights), key=lambda x: abs(x[1]),
                    reverse=True)[:20]
    for idx, w_val in ranked:
        print(f"  {feature_cols[idx]:<28} {w_val:>+10.4f}")
    print(f"  {'(intercept)':<28} {bias:>+10.4f}")

    # ---- Score test rows ----
```

```
    print(f"\n  Scoring all 120K test rows ...")
    by_user = defaultdict(list)
    for uid, candidates in test_users:
        for tid in candidates:
            row = (np.array(build_row(uid, tid)) - feat_mean) / feat_std
            score = float(row @ weights + bias)
            by_user[uid].append((tid, score))

    n_ones, n_total = _write_topk_submission(by_user, out_path)
    print(f"  {n_total:,} predictions  ({n_ones:,} rec / "
          f"{n_total-n_ones:,} not rec)")
    print(f"  Saved -> {os.path.basename(out_path)}")

    # ---- Pure iicf-only submission for comparison ----
    iicf_only_path = out_path.replace(".csv", "_iicf_only.csv")
    write_iicf_only_submission(test_users, iicf_scores, iicf_only_path)

    return out_path


def generate_stacked_submission_v5(test_users, v5_raw, v2_raw,
                                   iicf_variants,
                                   label_path, hw9_probs_path,
                                   past_preds_dir, out_path):
    """v5 stacker: v4 features minus ALS, with multiple item-item CF
    variants stacked as separate features.

    Decisions vs v4:
      - DROP ALS entirely.  At rank=80 the LR gave it -0.151/-0.117;
        at rank=20 it gave +0.04 (basically noise).  The feature has
        never meaningfully helped.
      - ADD two more iicf variants alongside the original (k30/m5):
          k50/m3 = more neighbors, looser similarity threshold
          k20/m10 = fewer neighbors, stricter threshold
        The meta-learner gets one rank + one score column per variant.

    Args:
      iicf_variants : dict {name: {(uid, tid): score}}, e.g.
                      {"iicf_a": {...}, "iicf_b": {...}, "iicf_c": {...}}
    """
    section("STACKED META-LEARNER v5 (iicf variants, no ALS)")

    if not os.path.exists(label_path):
        print(f"  test2_new.txt not found -- skipping")
        return None

    labels = load_test2_labels(label_path)
    print(f"  Loaded {len(labels):,} labels  "
          f"({sum(labels.values()):,} positives)")

    hw9 = load_hw9_probabilities(hw9_probs_path)
    if hw9 is not None:
        hw9_probs, hw9_models = hw9
    else:
        hw9_probs, hw9_models = None, []

    past_preds, past_cols = load_past_predictions(past_preds_dir)
    if past_preds is None:
        past_preds, past_cols = {}, []

    iicf_names = list(iicf_variants.keys())
    print(f"  iicf variants: {', '.join(iicf_names)}")

    # Per-user ranks
    v5_rank = _rank_normalize_per_user(v5_raw, test_users)
    v2_rank = _rank_normalize_per_user(v2_raw, test_users)
    iicf_ranks = {name: _rank_normalize_per_user(d, test_users)
                  for name, d in iicf_variants.items()}

    # Family consensus features
    families = {
        "v5b_hybrid": "v5b_hybrid_consensus",
        "v6_hybrid":  "v6_hybrid_consensus",
        "hw9":        "hw9_consensus",
        "v3":         "v3_consensus",
        "p1":         "p1_consensus",
        "hw8":        "hw8_consensus",
    }
    consensus_feats = {}
    consensus_cols  = []
```

```
    used_past_cols  = set()
    for prefix, col_name in families.items():
        cdict, members = _consensus_from_past(past_preds, past_cols, prefix)
        if cdict is None:
            continue
        consensus_feats[col_name] = cdict
        consensus_cols.append(col_name)
        used_past_cols.update(members)
    leftover_cols = [c for c in past_cols if c not in used_past_cols]

    # Isotonic calibration
    print("\n  Fitting isotonic calibrators on labeled subset ...")
    label_keys = list(labels.keys())
    label_groups = np.array([uid for uid, _ in label_keys])
    label_y      = np.array([labels[k] for k in label_keys],
                            dtype=np.float64)
    keep_mask = np.array([
        (k in v5_raw) and (k in v2_raw)
        and (hw9_probs is None or k in hw9_probs)
        for k in label_keys])
    keep_keys   = [k for k, m in zip(label_keys, keep_mask) if m]
    keep_groups = label_groups[keep_mask]
    keep_y      = label_y[keep_mask]

    v5_arr = np.array([v5_raw[k] for k in keep_keys], dtype=np.float64)
    _, iso_v5 = isotonic_calibrate_oof(v5_arr, keep_y, keep_groups)
    hw9_isos = {}
    if hw9_probs is not None:
        for m in hw9_models:
            arr = np.array([hw9_probs[k][m] for k in keep_keys],
                           dtype=np.float64)
            _, iso = isotonic_calibrate_oof(arr, keep_y, keep_groups)
            hw9_isos[m] = iso

    def calibrated_v5(uid, tid):
        return float(iso_v5.predict([v5_raw.get((uid, tid), 0.0)])[0])

    def calibrated_hw9(uid, tid, m):
        if hw9_probs is None:
            return 0.5
        row = hw9_probs.get((uid, tid))
        if row is None:
            return 0.5
        return float(hw9_isos[m].predict([row[m]])[0])

    # ---- Feature columns (NO ALS) ----
    rank_cols = ["v5_rank", "v2_rank"] + [f"{n}_rank" for n in iicf_names]
    score_cols = (["v5_iso", "v2_score"]
                  + [f"{n}_score" for n in iicf_names])
    feature_cols = (
        rank_cols
        + score_cols
        + [f"{m}_iso" for m in hw9_models]
        + consensus_cols
        + leftover_cols
    )
    print(f"\n  v5 feature set ({len(feature_cols)} cols):")
    print(f"    rank features : {len(rank_cols)} "
          f"(v5, v2, {len(iicf_names)} x iicf)")
    print(f"    raw scores    : {len(score_cols)}")
    print(f"    HW9 isotonic  : {len(hw9_models)}")
    print(f"    family cons.  : {len(consensus_cols)}")
    print(f"    leftover past : {len(leftover_cols)}")
    print(f"    ALS           : DROPPED (was -0.151/-0.117 at rank=80)")

    def build_row(uid, tid):
        row = [v5_rank.get((uid, tid), 0.5),
               v2_rank.get((uid, tid), 0.5)]
        for name in iicf_names:
            row.append(iicf_ranks[name].get((uid, tid), 0.5))
        row += [calibrated_v5(uid, tid),
                v2_raw.get((uid, tid), 0.0)]
        for name in iicf_names:
            row.append(iicf_variants[name].get((uid, tid), 0.0))
        for m in hw9_models:
            row.append(calibrated_hw9(uid, tid, m))
        for c in consensus_cols:
            row.append(consensus_feats[c].get((uid, tid), 0.5))
        votes = past_preds.get((uid, tid), {})
        for c in leftover_cols:
```

```
            row.append(float(votes.get(c, 0)))
        return row

    # Training matrix
    X_rows, y_rows, group_rows, missing = [], [], [], 0
    for (uid, tid), y in labels.items():
        if (uid, tid) not in v5_raw or (uid, tid) not in v2_raw:
            missing += 1
            continue
        if hw9_probs is not None and (uid, tid) not in hw9_probs:
            missing += 1
            continue
        X_rows.append(build_row(uid, tid))
        y_rows.append(y)
        group_rows.append(uid)
    if missing:
        print(f"\n  Skipped {missing:,} labeled rows (missing scores)")

    X = np.array(X_rows, dtype=np.float64)
    y = np.array(y_rows, dtype=np.float64)
    groups = np.array(group_rows)
    print(f"  Training matrix: {X.shape}  "
          f"({int(y.sum()):,} pos / {int(len(y)-y.sum()):,} neg, "
          f"{len(np.unique(groups)):,} unique users)")

    feat_mean = X.mean(axis=0)
    feat_std  = X.std(axis=0)
    feat_std[feat_std < 1e-8] = 1.0
    Xn = (X - feat_mean) / feat_std

    # GroupKFold sweep
    from sklearn.linear_model import LogisticRegression
    from sklearn.model_selection import GroupKFold
    from sklearn.metrics import roc_auc_score
    Cs = [0.001, 0.01, 0.1, 1.0, 10.0]
    gkf = GroupKFold(n_splits=5)

    print(f"\n  GroupKFold(5) by user -- NDCG@3 + AUC sweep")
    print(f"  {'C':>8}   {'NDCG@3':>8}   {'AUC':>8}")
    print(f"  {'-'*8}   {'-'*8}   {'-'*8}")
    sweep = []
    for C in Cs:
        ndcgs, aucs = [], []
        for tr, va in gkf.split(Xn, y, groups=groups):
            clf = LogisticRegression(C=C, max_iter=2000, random_state=SEED)
            clf.fit(Xn[tr], y[tr])
            p = clf.predict_proba(Xn[va])[:, 1]
            aucs.append(roc_auc_score(y[va], p))
            ndcgs.append(ndcg_at_k_grouped(y[va], p, groups[va], k=3))
        m_ndcg = float(np.mean(ndcgs))
        m_auc  = float(np.mean(aucs))
        sweep.append((C, m_ndcg, m_auc))
        print(f"  {C:>8.3f}   {m_ndcg:>8.4f}   {m_auc:>8.4f}")

    best_C, best_ndcg, best_auc = max(sweep, key=lambda r: r[1])
    print(f"\n  Best C = {best_C}  "
          f"(NDCG@3 = {best_ndcg:.4f}, AUC = {best_auc:.4f})")

    clf = LogisticRegression(C=best_C, max_iter=2000, random_state=SEED)
    clf.fit(Xn, y)
    weights = clf.coef_[0]
    bias    = float(clf.intercept_[0])

    print(f"\n  v5 LR weights (top 25 by abs):")
    print(f"  {'Feature':<28} {'Weight':>10}")
    print(f"  {'-'*28} {'-'*10}")
    ranked = sorted(enumerate(weights), key=lambda x: abs(x[1]),
                    reverse=True)[:25]
    for idx, w_val in ranked:
        print(f"  {feature_cols[idx]:<28} {w_val:>+10.4f}")
    print(f"  {'(intercept)':<28} {bias:>+10.4f}")

    # Score test rows
    print(f"\n  Scoring all 120K test rows ...")
    by_user = defaultdict(list)
    for uid, candidates in test_users:
        for tid in candidates:
            row = (np.array(build_row(uid, tid)) - feat_mean) / feat_std
            score = float(row @ weights + bias)
            by_user[uid].append((tid, score))
```

```python

    n_ones, n_total = _write_topk_submission(by_user, out_path)
    print(f"  {n_total:,} predictions  ({n_ones:,} rec / "
          f"{n_total-n_ones:,} not rec)")
    print(f"  Saved -> {os.path.basename(out_path)}")

    return out_path


def generate_stacked_submission(test_users, v5_raw, v2_raw,
                                label_path, hw9_probs_path,
                                past_preds_dir, out_path,
                                als_scores=None):
    """Train a meta-learner on test2_new.txt labels using all available
    score columns, then apply to all 120K test rows and write submission.

    Returns the output path on success, None if labels are missing.
    """
    section("STACKED META-LEARNER")

    if not os.path.exists(label_path):
        print(f"  test2_new.txt not found at {label_path} -- skipping stacking")
        return None

    labels = load_test2_labels(label_path)
    print(f"  Loaded {len(labels):,} labels  "
          f"({sum(labels.values()):,} positives)")

    # Optional: load HW9 PySpark probabilities
    hw9 = load_hw9_probabilities(hw9_probs_path)
    if hw9 is not None:
        hw9_probs, hw9_models = hw9
    else:
        hw9_probs, hw9_models = None, []
        print(f"\n  hw9_probabilities.csv not found at:")
        print(f"    {hw9_probs_path}")
        print(f"  Stacking on (v5, v2) only from this run -- run HW9 first")
        print(f"  to add 7 more PySpark columns.")

    # Optional: load past Kaggle submissions as binary feature columns
    past_preds, past_cols = load_past_predictions(past_preds_dir)
    if past_preds is None:
        print(f"\n  No past predictions in {past_preds_dir}")

    # Build feature matrix for the LABELED set
    has_als      = als_scores is not None
    als_col      = ["als_score"] if has_als else []
    feature_cols = (["v5_score", "v2_score"] + als_col
                    + list(hw9_models) + list(past_cols))
    print(f"\n  Stacking features ({len(feature_cols)} total): "
          f"v5+v2 + {len(als_col)} ALS + {len(hw9_models)} HW9 "
          f"+ {len(past_cols)} past")

    X_rows, y_rows, missing = [], [], 0
    for (uid, tid), y in labels.items():
        v5 = v5_raw.get((uid, tid))
        v2 = v2_raw.get((uid, tid))
        if v5 is None or v2 is None:
            missing += 1
            continue
        row = [v5, v2]
        if has_als:
            row.append(als_scores.get((uid, tid), 0.0))
        if hw9_probs is not None:
            hw9_row = hw9_probs.get((uid, tid))
            if hw9_row is None:
                missing += 1
                continue
            row += [hw9_row[m] for m in hw9_models]
        if past_preds is not None:
            past_row = past_preds.get((uid, tid), {})
            row += [float(past_row.get(c, 0)) for c in past_cols]
        X_rows.append(row)
        y_rows.append(y)

    if missing:
        print(f"  Skipped {missing:,} labeled rows (missing scores)")

    X = np.array(X_rows, dtype=np.float64)
    y = np.array(y_rows, dtype=np.float64)
```

```
    print(f"  Training matrix: {X.shape}  "
          f"({int(y.sum()):,} pos / {int(len(y)-y.sum()):,} neg)")

    # Standardize for numerical stability of the LR
    feat_mean = X.mean(axis=0)
    feat_std  = X.std(axis=0)
    feat_std[feat_std < 1e-8] = 1.0
    Xn = (X - feat_mean) / feat_std

    # Fit LR -- prefer sklearn if present
    weights = None
    bias    = None
    try:
        from sklearn.linear_model import LogisticRegressionCV
        clf = LogisticRegressionCV(Cs=10, cv=5, scoring="roc_auc",
                                   max_iter=2000, n_jobs=-1,
                                   random_state=SEED)
        clf.fit(Xn, y)
        weights = clf.coef_[0]
        bias    = float(clf.intercept_[0])
        cv_auc  = float(clf.scores_[1.0].mean(axis=0).max())
        print(f"\n  sklearn LogisticRegressionCV  "
              f"(best C={clf.C_[0]:.4f}, 5-fold CV AUC={cv_auc:.4f})")
    except ImportError:
        print(f"\n  sklearn not available -- using hand-coded LR")
        w, b   = fit_logistic_regression(Xn, y, n_iter=3000, lr=0.05, l2=0.01)
        weights = w
        bias    = b

    # Print learned weights -- which signals matter
    print(f"\n  Meta-learner weights (on standardized features):")
    print(f"  {'Feature':<20} {'Weight':>10}")
    print(f"  {'-'*20} {'-'*10}")
    ranked = sorted(enumerate(weights), key=lambda x: abs(x[1]), reverse=True)
    for idx, w_val in ranked:
        print(f"  {feature_cols[idx]:<20} {w_val:>+10.4f}")
    print(f"  {'(intercept)':<20} {bias:>+10.4f}")

    # Score every test row, group by user, pick top-3
    print(f"\n  Scoring all 120K test rows ...")
    by_user = defaultdict(list)
    for uid, candidates in test_users:
        for tid in candidates:
            row = [v5_raw.get((uid, tid), 0.0),
                   v2_raw.get((uid, tid), 0.0)]
            if has_als:
                row.append(als_scores.get((uid, tid), 0.0))
            if hw9_probs is not None:
                hw9_row = hw9_probs.get((uid, tid))
                if hw9_row is None:
                    row += [0.5] * len(hw9_models)   # neutral fallback
                else:
                    row += [hw9_row[m] for m in hw9_models]
            if past_preds is not None:
                past_row = past_preds.get((uid, tid), {})
                row += [float(past_row.get(c, 0)) for c in past_cols]
            row_n = (np.array(row) - feat_mean) / feat_std
            score = float(row_n @ weights + bias)
            by_user[uid].append((tid, score))

    # Top-3 per user
    n_ones, n_total = 0, 0
    with open(out_path, "w") as f:
        f.write("TrackID,Predictor\n")
        for uid, items in by_user.items():
            top3 = set(t for t, _ in sorted(items, key=lambda x: x[1],
                                            reverse=True)[:3])
            for tid, _ in items:
                pred = 1 if tid in top3 else 0
                f.write(f"{uid}_{tid},{pred}\n")
                n_ones  += pred
                n_total += 1

    print(f"  {n_total:,} predictions  "
          f"({n_ones:,} rec / {n_total-n_ones:,} not rec)")
    print(f"  Saved -> {os.path.basename(out_path)}")
    return out_path


# ==================================================================
```

```python
# 13. Main
# ==================================================================

def main():
    tee = Tee(RESULTS_FILE)
    sys.stdout = tee
    t_total = time.time()

    torch.manual_seed(SEED)
    random.seed(SEED)
    np.random.seed(SEED)

    section("EE627 FINAL PROJECT - Music Recommender")
    print(f"  Log file : {RESULTS_FILE}")
    print(f"  Started  : {time.strftime('%Y-%m-%d %H:%M:%S')}")
    print(f"  Device   : {DEVICE}")
    if torch.cuda.is_available():
        print(f"  GPU      : {torch.cuda.get_device_name(0)}")

    # ---- Load data ----
    section("LOADING RAW DATA")
    user_ratings = parse_training(os.path.join(DATA_DIR, "trainItem2.txt"))
    track_meta   = parse_tracks  (os.path.join(DATA_DIR, "trackData2.txt"))
    album_meta   = parse_albums  (os.path.join(DATA_DIR, "albumData2.txt"))
    test_users   = parse_test    (os.path.join(DATA_DIR, "testItem2.txt"))

    # ---- Enrich + auxiliary structures ----
    section("ENRICHMENT & AUXILIARY STRUCTURES")
    enriched      = enrich_tracks_with_albums(track_meta, album_meta)
    global_stats  = build_global_item_stats(user_ratings)
    album_to_tracks, artist_to_tracks = build_track_mappings(enriched)

    # ---- Matrix factorization (#10) ----
    section("MATRIX FACTORIZATION (GPU)")
    mf_uid_map, mf_iid_map, mf_user_f, mf_item_f = train_mf_embeddings(
        user_ratings, MF_FACTORS, MF_EPOCHS, MF_LR, MF_BATCH)

    # ---- User split for validation ----
    section("BUILDING TRAINING & VALIDATION DATA")
    rng = random.Random(SEED)
    all_uids = list(user_ratings.keys())
    rng.shuffle(all_uids)
    split_pt    = int(len(all_uids) * 0.9)
    train_uids  = set(all_uids[:split_pt])
    val_uids    = set(all_uids[split_pt:])
    print(f"  Train users: {len(train_uids):,}  |  "
          f"Val users: {len(val_uids):,}")

    if os.path.exists(FEATURES_CACHE):
        print(f"\n  Loading cached features from "
              f"{os.path.basename(FEATURES_CACHE)} ...")
        t_feat = time.time()
        z = np.load(FEATURES_CACHE, allow_pickle=True)
        train_X    = z["train_X"]
        train_R    = z["train_R"]
        val_X      = z["val_X"]
        val_R      = z["val_R"]
        feat_mean  = z["feat_mean"]
        feat_std   = z["feat_std"]
        val_groups = z["val_groups"].tolist()
        print(f"  Train X: {train_X.shape}  |  Val X: {val_X.shape}  "
              f"[{elapsed(t_feat)}]")
    else:
        train_groups = build_groups(user_ratings, enriched, train_uids,
                                    GROUPS_PER_USER, SEED)
        val_groups   = build_groups(user_ratings, enriched, val_uids,
                                    10, SEED + 1)
        print(f"  Train groups: {len(train_groups):,}  |  "
              f"Val groups: {len(val_groups):,}")

        print("\n  Featurizing training groups ...")
        train_X, train_R = featurize_groups(
            train_groups, user_ratings, enriched, global_stats,
            album_to_tracks, artist_to_tracks,
            mf_uid_map, mf_iid_map, mf_user_f, mf_item_f)

        print("  Featurizing validation groups ...")
        val_X, val_R = featurize_groups(
            val_groups, user_ratings, enriched, global_stats,
            album_to_tracks, artist_to_tracks,
```

```
            mf_uid_map, mf_iid_map, mf_user_f, mf_item_f)

        # ---- Normalize (#4) ----
        feat_mean, feat_std = compute_norm_stats(train_X)
        train_X = normalize(train_X, feat_mean, feat_std)
        val_X   = normalize(val_X,   feat_mean, feat_std)

        np.savez(FEATURES_CACHE,
                 train_X=train_X, train_R=train_R,
                 val_X=val_X, val_R=val_R,
                 feat_mean=feat_mean, feat_std=feat_std,
                 val_groups=np.array(val_groups, dtype=object))
        print(f"  Cached -> {os.path.basename(FEATURES_CACHE)}")

    # ---- v2 baseline AUC (sanity check) ----
    section("V2 HEURISTIC BASELINE")
    v2_auc = v2_heuristic_val_auc(user_ratings, enriched, val_groups)
    print(f"  v2 heuristic val AUC: {v2_auc:.4f}  (expected Kaggle ~0.871)")

    # ---- Train neural ranker ----
    section("TRAINING RANKING MODEL (BPR)")
    model, best_auc = train_ranking_model(train_X, train_R, val_X, val_R)

    # ---- Submissions ----
    section("GENERATING SUBMISSIONS")

    # 1. Pure v5 neural network  (returns binary results + raw scores)
    v5_results, v5_raw_scores = predict_test(
        model, user_ratings, enriched, test_users,
        feat_mean, feat_std,
        global_stats, album_to_tracks, artist_to_tracks,
        mf_uid_map, mf_iid_map, mf_user_f, mf_item_f,
        os.path.join(OUTPUT_DIR, "submission_final_v5_nn.csv"),
        "v5 BPR Neural Net")

    # 2. v2 heuristic baseline
    predict_v2_baseline(
        user_ratings, enriched, test_users,
        os.path.join(OUTPUT_DIR, "submission_final_v2.csv"))

    # Build v2 score lookup for stacking
    v2_raw_scores = build_v2_score_dict(test_users, user_ratings, enriched)

    # 3. Confidence-gated hybrid sweep -- gap10 = production submission
    generate_hybrid_submissions(
        test_users, user_ratings, enriched,
        v5_results, HYBRID_THRESHOLDS, OUTPUT_DIR)

    # 4. PySpark ALS -- another meta-learner input column.
    #    Pre-load the labeled-set keys so ALS knows which (uid, tid) pairs
    #    we'll need scored for the meta-learner training matrix.
    als_label_keys = []
    if os.path.exists(LABEL_FILE):
        with open(LABEL_FILE, "r") as f:
            for line in f:
                parts = line.strip().split("|")
                if len(parts) == 3:
                    als_label_keys.append((int(parts[0]), int(parts[1])))
    als_scores = train_als_and_score(
        user_ratings, test_users, als_label_keys,
        os.path.join(OUTPUT_DIR, "submission_final_als.csv"))

    # 5. Stacked meta-learner submission (v1 -- 0.918 on Kaggle)
    stacked_path = generate_stacked_submission(
        test_users, v5_raw_scores, v2_raw_scores,
        LABEL_FILE, HW9_PROBS_FILE, PAST_PREDS_DIR,
        os.path.join(OUTPUT_DIR, "submission_final_stacked.csv"),
        als_scores=als_scores)

    # 6. Stacked meta-learner v2 -- per-user ranks, family consensus,
    #    GroupKFold CV, NNLS sanity baseline.
    stacked_v2_path = generate_stacked_submission_v2(
        test_users, v5_raw_scores, v2_raw_scores,
        LABEL_FILE, HW9_PROBS_FILE, PAST_PREDS_DIR,
        os.path.join(OUTPUT_DIR, "submission_final_stacked_v2.csv"),
        als_scores=als_scores)

    # 7. Stacked meta-learner v3 -- LightGBM LambdaRank base model,
    #    isotonic calibration of HW9/v5, NDCG@3 grouped CV tuning.
    #    Writes 5 submissions: V2ref, A_lgbm, B_iso, C_ndcg, and ALL.
```

```
    stacked_v3_path = generate_stacked_submission_v3(
        test_users, v5_raw_scores, v2_raw_scores,
        train_X, val_X,
        user_ratings, enriched,
        feat_mean, feat_std,
        global_stats, album_to_tracks, artist_to_tracks,
        mf_uid_map, mf_iid_map, mf_user_f, mf_item_f,
        LABEL_FILE, HW9_PROBS_FILE, PAST_PREDS_DIR,
        os.path.join(OUTPUT_DIR, "submission_final_stacked_v3.csv"),
        als_scores=als_scores)

    # 8. Item-item CF base model (genuinely orthogonal signal)
    section("ITEM-ITEM COLLABORATIVE FILTERING")
    iicf_scores = compute_or_load_iicf(test_users, user_ratings)

    # 9. Stacked meta-learner v4 -- v3 ALL features + item-item CF.
    stacked_v4_path = generate_stacked_submission_v4(
        test_users, v5_raw_scores, v2_raw_scores, iicf_scores,
        LABEL_FILE, HW9_PROBS_FILE, PAST_PREDS_DIR,
        os.path.join(OUTPUT_DIR, "submission_final_stacked_v4.csv"),
        als_scores=als_scores)

    # 10. Item-item CF tuning variants for v5
    section("ITEM-ITEM CF TUNING VARIANTS (for v5)")
    print("  Building shared item index once for all variants ...")
    t_idx = time.time()
    iicf_index = build_item_inverted_index(user_ratings)
    print(f"  {len(iicf_index[0]):,} items indexed  [{elapsed(t_idx)}]")
    iicf_variants_dict = {}
    for name, k, m in IICF_VARIANTS:
        print(f"\n  Variant {name}: topk={k}, min_co={m}")
        iicf_variants_dict[name] = compute_or_load_iicf(
            test_users, user_ratings,
            topk=k, min_co=m, item_index_cache=iicf_index)

    # 11. Stacked meta-learner v5 -- iicf variants stacked, ALS dropped.
    stacked_v5_path = generate_stacked_submission_v5(
        test_users, v5_raw_scores, v2_raw_scores,
        iicf_variants_dict,
        LABEL_FILE, HW9_PROBS_FILE, PAST_PREDS_DIR,
        os.path.join(OUTPUT_DIR, "submission_final_stacked_v5_real.csv"))

    # ---- Summary ----
    section("FINAL SUMMARY")
    print(f"  v2 heuristic val AUC : {v2_auc:.4f}  (Kaggle baseline 0.871)")
    print(f"  v5 BPR-NN  val AUC   : {best_auc:.4f}")
    print(f"  Delta                : {best_auc - v2_auc:+.4f}")
    print()
    print(f"  Submissions written:")
    print(f"    submission_final_v2.csv                  v2 heuristic alone")
    print(f"    submission_final_v5_nn.csv               pure neural net")
    print(f"    submission_final_v5_hybrid_gap{{N}}.csv    hybrid sweep")
    if stacked_path is not None:
        print(f"    submission_final_stacked.csv             stacked meta-learner v1 (0.918)")
    if stacked_v2_path is not None:
        print(f"    submission_final_stacked_v2.csv          stacked v2 (rank+consensus+GroupKFold)")
        print(f"    submission_final_stacked_v2_nnls.csv     NNLS sanity baseline")
    if stacked_v3_path is not None:
        print(f"    submission_final_stacked_v3.csv          stacked v3 (LightGBM+Isotonic+NDCG@3)")
        print(f"    submission_final_stacked_v3_A_lgbm.csv   ablation: +LightGBM only")
        print(f"    submission_final_stacked_v3_B_iso.csv    ablation: +Isotonic only")
        print(f"    submission_final_stacked_v3_C_ndcg.csv   ablation: +NDCG@3 tuning only")
        print(f"    submission_final_stacked_v3_V2ref.csv    ablation: V2 reference re-fit")
    if stacked_v4_path is not None:
        print(f"    submission_final_stacked_v4.csv          stacked v4 (item-item CF + v3) -- 0.922")
        print(f"    submission_final_stacked_v4_iicf_only.csv  pure item-item CF top-3 -- 0.781")
    if stacked_v5_path is not None:
        print(f"    submission_final_stacked_v5_real.csv     stacked v5 (3 iicf variants, no ALS)")
    print()
    print(f"  Recommended Kaggle submissions:")
    print(f"    submission_final_v5_hybrid_gap10.csv     "
          f"(scored 0.911 on midterm Kaggle)")
    if stacked_path is not None:
        print(f"    submission_final_stacked.csv             "
              f"(v1 meta-learner -- scored 0.918)")
    if stacked_v2_path is not None:
        print(f"    submission_final_stacked_v2.csv          "
              f"(v2 meta-learner -- scored 0.919)")
    if stacked_v3_path is not None:
        print(f"    submission_final_stacked_v3.csv          "
```

```
            f"(v3 meta-learner -- LightGBM+Isotonic+NDCG@3)")
    if stacked_v4_path is not None:
        print(f"    submission_final_stacked_v4.csv         "
            f"(v4 meta-learner -- scored 0.922)")
    if stacked_v5_path is not None:
        print(f"    submission_final_stacked_v5_real.csv    "
            f"(v5 meta-learner -- 3 iicf variants, no ALS)")
    print()
    print(f"  Total wall time : {elapsed(t_total)}")

    tee.close()


if __name__ == "__main__":
    main()
```

Listing 1: `EE627_Final_Eschete.py` — full pipeline: MF, BPR neural ranker, ALS, item-item CF, and the v1–v5 stackers.